

5

Defining Classes

In this chapter we will explore the heart of the Java language – classes. Classes specify the objects you use in object-oriented programming. These form the basic building blocks of any Java program, as we saw in Chapter 1. Every program in Java involves classes, since the code for a program can only appear within a class definition.

We will now explore the details of how a class definition is put together, how to create your own classes and how to use classes to solve your own computing problems. And in the next chapter we'll extend this to look at how object-oriented programming helps us work with related classes.

By the end of this chapter you will have learned:

- ❑ What a class is, and how you define one.
- ❑ How to implement class constructors.
- ❑ How to define class methods.
- ❑ What method overloading is.
- ❑ What a recursive method is and how it works.
- ❑ How to create objects of a class.
- ❑ What packages are and how you can create and use them.
- ❑ What access attributes are and how you should use them in your class definitions.
- ❑ When you should add the `finalize()` method to a class.
- ❑ What native methods are.

What is a Class?

As you saw in Chapter 1, a class is a prescription for a particular kind of object – it defines a new **type**. We can use the class definition to create objects of that class type, that is, to create objects that incorporate all the components specified as belonging to that class.

In case that's too abstract, look back to the last chapter, where we used the `String` class. This is a comprehensive definition for a string object, with all the operations you are likely to need built in. This makes `String` objects indispensable and string handling within a program easy.

The `String` class lies towards one end of a spectrum in terms of complexity in a class. The `String` class is intended to be usable in any program. It includes facilities and capabilities for operating on `String` objects to cover virtually all circumstances in which you are likely to use strings. In most cases your own classes won't need to be this elaborate. You will typically be defining a class to suit your particular application. A very simple class for instance, a `Plane` or a `Person`, may well represent objects that can potentially be very complicated, if that fulfils your needs. A `Person` object might just contain a name, address, and phone number for example if you are just implementing an address book. In another context, in a payroll program perhaps, you might need to represent a `Person` with a whole host of properties, such as age, marital status, length of service, job code, pay rate, and so on. It all depends on what you intend to do with objects of your class.

In essence a class definition is very simple. There are just two kinds of things that you can include in a class definition:

- ❑ **Fields**
These are variables that store data items that typically differentiate one object of the class from another. They are also referred to as **data members** of a class.
- ❑ **Methods**
These define the operations you can perform for the class – so they determine what you can do to, or with, objects of the class. Methods typically operate on the fields – the variables of the class.

The fields in a class definition can be of any of the basic types, or they can be references to objects of any class type, including the one that you are defining.

The methods in a class definition are named, self-contained blocks of code that typically operate on the variables that appear in the class definition. Note though, that this doesn't necessarily have to be the case, as you might have guessed from the `main()` methods we have written in all our examples up to now.

Variables in a Class Definition

An object of a class is also referred to as an **instance** of that class. When you create an object, the object will contain all the variables that were included in the class definition. However, the variables in a class definition are not all the same – there are two kinds.

One kind of variable in a class is associated with each object uniquely – each instance of the class will have its own copy of each of these variables, with its own value assigned. These differentiate one object from another, giving an object its individuality – the particular name, address, and telephone number in a given `Person` object for instance. These are referred to as **instance variables**.

The other kind of class variable is associated with the class, and is shared by all objects of the class. There is only one copy of each of these kinds of variables no matter how many class objects are created, and they exist even if no objects of the class have been created. This kind of variable is referred to as a **class variable** because the variable belongs to the class, not to any particular object, although as we have said, all objects of the class will share it. These variables are also referred to as **static fields** because, as we will see, you use the keyword `static` when you declare them.

Because this is extremely important to understand, let's summarize the two kinds of variables that you can include in your classes:

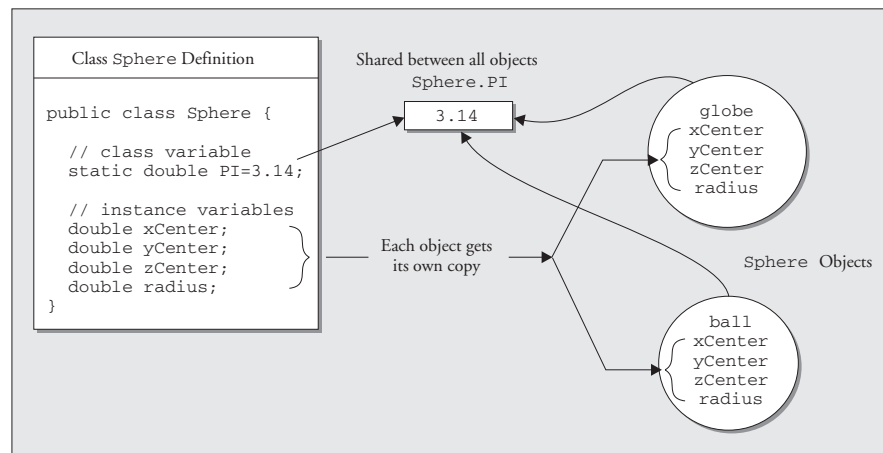
❑ **Instance variables**

Each object of the class will have its own copy of each of the instance variables that appear in the class definition. Each object will have its own values for each instance variable. The name 'instance variable' originates from the fact that an object is an 'instance' or an occurrence of a class and the values stored in the instance variables for the object differentiate the object from others of the same class type. An instance variable is declared within the class definition in the usual way, with a type name and a variable name, and can have an initial value specified.

❑ **Class variables**

A given class will only have one copy of each of its class variables, and these will be shared between all the objects of the class. The class variables exist even if no objects of the class have been created. They belong to the class, and they can be referenced by any object or class, not just instances of that class. If the value of a class variable is changed, the new value is available in all the objects of the class. This is quite different from instance variables where changing a value for one object does not affect the values in other objects. A class variable must be declared using the keyword `static` preceding the type name.

Look at the following diagram, which illustrates the difference between the two:



This shows a schematic of a class `Sphere` with one class variable `PI`, and four instance variables, `radius`, `xCenter`, `yCenter`, and `zCenter`. Each of the objects, `globe` and `ball`, will have their own variables, `radius`, `xCenter`, `yCenter`, and `zCenter`, but both will share a single copy of the class variable `PI`.

Why would you need two kinds of variables in a class definition? The instance variables are clearly necessary since they are the parameters that distinguish a particular object. The radius and the coordinates of the center of the sphere are fundamental to determining how big a particular `Sphere` object is, and where it is in space. However, although the variable `PI` is a fundamental parameter for a sphere – to calculate the volume for example – it would be wasteful to store a value for `PI` in every object, since it is always the same. Incidentally, it is also available from the standard class `Math` so it is somewhat superfluous in this case, but you get the general idea. So one use for class variables is to hold constant values such as π that are common to all objects of the class.

Another use for class variables is to track data values that are common to all objects of a class, and that need to be available even when no objects have been defined. For example, if you wanted to keep a count of how many objects of a class have been created in your program, you would define the variable storing the count as a class variable. It would be essential to use a class variable, because you would still want to be able to use your `count` variable even when no objects have been declared.

Methods in a Class Definition

The methods that you define for a class provide the actions that can be carried out using the variables specified in the class definition.

Analogous to the variables in a class definition, there are two varieties of methods – **instance methods** and **class methods**. You can execute class methods even when no objects of a class exist, whereas instance methods can only be executed in relation to a particular object, so if no objects exist, there are no instance methods to be executed. Again, like class variables, class methods are declared using the keyword `static` so they are sometimes referred to as **static methods**. We saw in the previous chapter that the `valueOf()` method is a static member of the `String` class.

Since class methods can be executed when there are no objects in existence, they cannot refer to instance variables. This is quite sensible if you think about it – trying to operate with variables that might not exist is bound to cause trouble. In fact the Java compiler won't let you try. If you reference an instance variable in the code for a class method, it won't compile – you'll just get an error message. The method `main()`, where execution of a Java application starts, must always be declared as static, as you have seen. The reason for this should be apparent by now. Before an application starts execution, no objects exist, so in order to start execution, you need a method that is executable even though there are no objects – a static method therefore.

The class `Sphere` might well have an instance method `volume()` to calculate the volume of a particular object. It might also have a class method `objectCount()` to return the current count of how many objects of type `Sphere` have been created. If no objects exist, you could still call this method and get the count 0.

Note that, although instance methods are specific to objects of a class, there is only ever one copy of an instance method in memory that is shared by all objects of the class, as it would be extremely expensive to replicate all the instance methods for each object. There is a special mechanism that ensures that, each time you call a method the codes executes in a manner that is specific to an object, but we will defer exploring this until a little later in this chapter.

Apart from making the method `main()`, perhaps the most common use for class methods is when a class is just used to contain a bunch of utility methods, rather than as a specification for objects. All executable code in Java has to be within a class, but there are lots of general-purpose functions that you need that don't necessarily have an object association – calculating a square root, for instance, or generating a random number. For example, the mathematical functions that are implemented as class methods in the standard class `Math`, don't relate to class objects at all – they operate on values of the basic types. You don't need objects of type `Math`, you just want to use the methods from time to time, and you can do this as we saw in Chapter 2. The class `Math` also contains some class variables containing useful mathematical constants such as e and π .

Accessing Variables and Methods

You will often want to access variables and methods, defined within a class, from outside it. We will see later that it is possible to declare class members with restrictions on accessing them from outside, but let's cover the principles that apply where the members are accessible. We need to consider accessing static members and instance members separately.

You can access a static member of a class using the class name, followed by a period, followed by the member name. With a class method you will also need to supply the parentheses enclosing any arguments to the method after the method name. The period here is called the dot operator. So, if you wanted to calculate the square root of π you could access the class method `sqrt()` and the class variable `PI` that are defined in the `Math` class as follows:

```
double rootPi = Math.sqrt(Math.PI);
```

This shows how you call a static method – you just prefix it with the class name and put the dot operator between them. We also reference the static data member, `PI`, in the same way – as `Math.PI`. If you have a reference to an object of a class type available, then you can also use that to access a static member of a class method. You just use the variable name, followed by the dot operator, followed by the member name.

Instance variables and methods can only be called using an object reference, as by definition they relate to a particular object. The syntax is exactly the same as we have outlined for static members. You put the name of the variable referencing the object followed by a period, followed by the member name. To use a method `volume()` that has been declared as an instance method in the `Sphere` class, you might write:

```
double ballVolume = ball.volume();
```

Here the variable `ball` is of type `Sphere` and it contains a reference to an object of this type. We call its `volume()` method that calculates the volume of the `ball` object, and the result that is returned is stored in the variable, `ballVolume`.

Defining Classes

To define a class you use the keyword `class` followed by the name of the class, followed by a pair of braces enclosing the details of the definition. Let's consider a concrete example to see how this works in practice. The definition of the `Sphere` class we mentioned earlier could be:

```
class Sphere {
    static final double PI = 3.14; // Class variable that has a fixed value
    static int count = 0;          // Class variable to count objects

    // Instance variables
    double radius;                 // Radius of a sphere

    double xCenter;                // 3D coordinates
    double yCenter;                // of the center
    double zCenter;                // of a sphere

    // Plus the rest of the class definition...
}
```

You name a class using an identifier of the same sort you've been using for variables. By convention though, class names in Java begin with a capital letter so our class name is `Sphere` with a capital `S`. If you adopt this approach, you will be consistent with most of the code you come across. You could enter this sourcecode and save it as the file `Sphere.java`. We will be building on this class, and using it in a working example, a little later in this chapter.

The keyword `static` in the first line of the definition specifies the variable `PI` as a class variable rather than an instance variable. The variable `PI` is also initialized with the value 3.14. The keyword `final` tells the compiler that you do not want the value of this variable to be changed, so the compiler will check that this variable is not modified anywhere in your program. Obviously this is a very poor value for π . You would normally use `Math.PI` – which is defined to twenty decimal places, close enough for most purposes.

Whenever you want to fix the value stored in a variable, that is, make it a constant, you just need to declare the variable with the keyword `final` and specify its initial value. By convention, constants have names in capital letters.

The next variable, `count`, is also declared with the keyword `static`. All objects of the `Sphere` class will share one copy of `count`, and one of `PI`. We have initialized the variable `count` to 0, but since it is not declared with the keyword `final`, we can change its value.

The next four variables in the class definition are instance variables, as they don't have the keyword `static` applied to them. Each object of the class will have its own separate set of these variables storing the radius and the coordinates of the center of the sphere. Although we haven't put initial values for these variables here, we could do so if we wanted. If you don't specify an initial value, a default value will be assigned automatically when the object is created. Fields that are of numeric types will be initialized with zero, fields of type `char` will be initialized with `'\u000'`, and fields that store class references or references to arrays will be initialized with `null`.

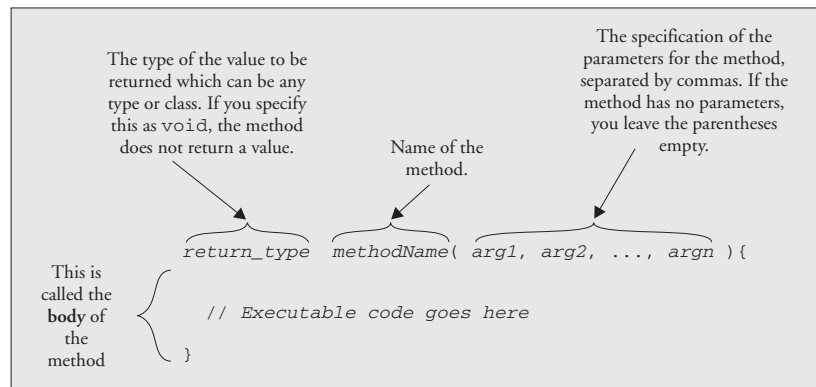
There has to be something missing from the definition of the `Sphere` class – there is no way to set the value of `radius` and the other instance variables once a particular `Sphere` object is created. There is nothing to update the value of `count` either. Adding these things to the class definition involves using methods, so we now need to look at how a method is put together.

Defining Methods

We have been producing versions of the method `main()` since Chapter 1, so you already have an idea of how a method is constructed. Nonetheless, we will go through from the beginning to make sure everything is clear.

We'll start with the fundamental concepts. A method is a self-contained block of code that has a name, and has the property that it is reusable – the same method can be executed from as many different points in a program as you require. Methods also serve to break up large and complex calculations that might involve many lines of code into more manageable chunks. A method is executed by calling its name, as we will see, and the method may or may not return a value. Methods that do not return a value are called in a statement that just does the call. Methods that do return a value are usually called from within an expression, and the value that is returned by such a method is used in the evaluation of the expression.

The basic structure of a method is shown below.



When you specify the return type for a method, you are defining the type for the value that will be returned by the method when you execute it. The method must always return a value of this type. To define a method that does not return a value, you specify the return type as `void`. Something called an **access attribute** can optionally precede the return type in a method definition, but we will defer looking into this until later in this chapter.

The parameters to a method appear in its definition between parentheses following the method name. These specify what information is to be passed to the method when you execute it. Your methods do not have to have parameters specified. A method that does not require any information to be passed to it when it is executed has an empty pair of parentheses after the name.

Running from a Method

To return a value from a method when its execution is complete you use a `return` statement, for example:

```
return return_value;    // Return a value from a method
```

After executing the `return` statement, the program continues from the point where the method was called. The value, `return_value`, that is returned by the method can be any expression that produces a value of the type specified for the return value in the declaration of the method. Methods that return a value – that is methods declared with a return type other than `void` – must always finish by executing a `return` statement that returns a value of the appropriate type. Note, though, that you can put several `return` statements within a method if the logic requires this. If a method does not return a value, you can just use the keyword `return` by itself to end execution of the method:

```
return;    // Return from a method
```

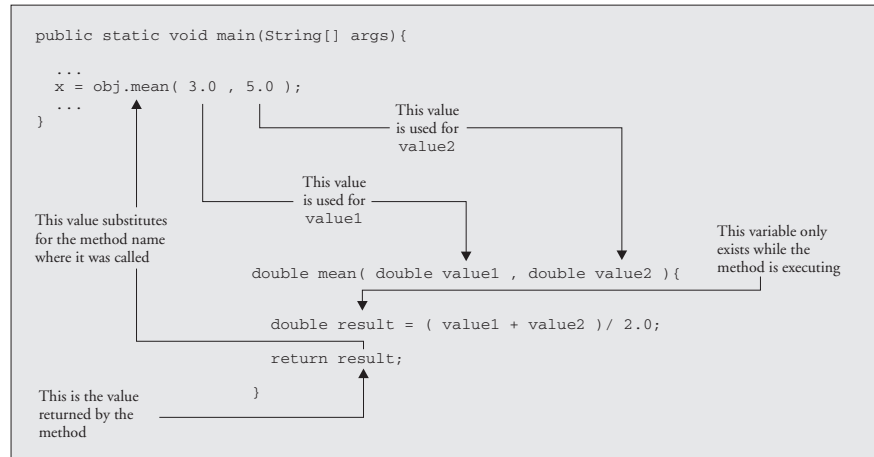
Note that, for methods that do not return a value, falling through the closing brace enclosing the body of the method is equivalent to executing a `return` statement.

The Parameter List

The **parameter list** appears between the parentheses following the method name. This specifies the type of each value that can be passed as an argument to the method, and the variable name that is to be used in the body of the method to refer to each value passed. The difference between a **parameter** and an **argument** is sometimes confusing because people often, incorrectly, use them interchangeably. We will try to differentiate them consistently, as follows:

- ❑ A parameter has a name and appears in the parameter list in the definition of a method. A parameter defines the type of value that can be passed to the method, and the name that is used to reference it within the code for the method.
- ❑ An argument is a value that is passed to a method when it is executed, and the value of the argument is referenced by the parameter name during execution of the method.

This is illustrated in the following diagram.



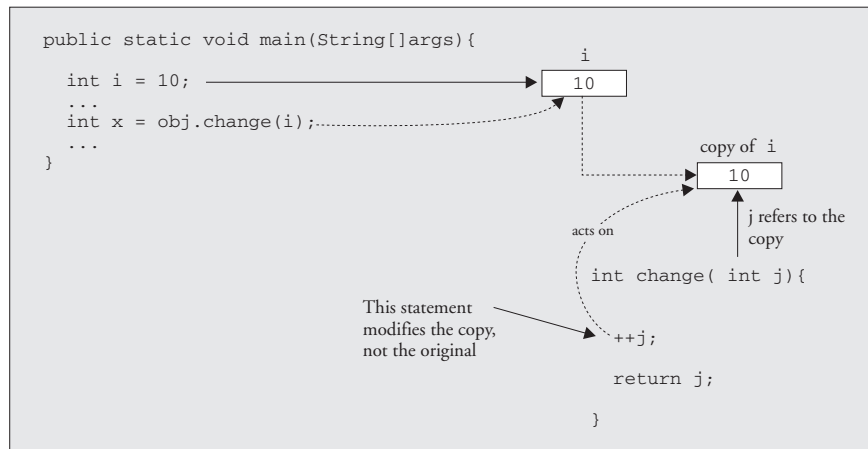
Here we have the definition of a method `mean()`. This can only appear within the definition of a class, but the rest of the class definition has been omitted so as not to clutter up the diagram. You can see that the method has two parameters, `value1`, and `value2`, both of which are of type `double`, that are used to refer to the arguments 3.0 and 5.0 respectively within the body of the method. Since this method has not been defined as `static`, you can only call it for an object of the class. We call `mean()` in our example for the object, `obj`.

When you call the method from another method (from `main()` in this case, but it could be from some other method), the values of the arguments passed are the initial values assigned to the corresponding parameters. You can use any expression you like for an argument when you call a method, as long as the value it produces is of the same type as the corresponding parameter in the definition of the method. With our method `mean()`, both parameters are of type `double`, so both argument values must always be of type `double`.

The method `mean()` declares the variable `result`, which only exists within the body of the method. The variable is created each time you execute the method and it is destroyed when execution of the method ends. All the variables that you declare within the body of a method are local to the method, and are only around while the method is being executed. Variables declared within a method are called **local variables** because they are local to the method. The scope of a local variable is as we discussed in Chapter 2, and local variables are not initialized automatically. If you want your local variables to have initial values you must supply the initial value when you declare them.

How Argument Values Are Passed to a Method

You need to be clear about how your argument values are passed to a method, otherwise you may run into problems. In Java, all argument values that belong to one of the basic types are transferred to a method using what is called the **pass-by-value** mechanism. How this works is illustrated below.



This just means that for each argument value that you pass to a method, a copy is made, and it is the copy that is passed to the method and referenced through the parameter name, not the original value. This implies that if you use a variable of any of the basic types as an argument, the method cannot modify the value of this variable in the calling program. In the example shown, the method `change()` will modify the copy of `i` that is created automatically, so the value of `j` that is returned will be 11 and this will be stored in `x`. However, the original value of `i` will remain at 10.

While the pass-by-value mechanism applies to all types of arguments, the effect for objects is different from that for variables of the basic types. You can change an object, as we shall see a little later in this chapter, because a copy of a reference to the object is passed to the method, not a copy of the object itself.

Final Parameters

You can specify any method parameter as `final`. This has the effect of preventing modification of any argument value that is substituted for the parameter when you call the method. The compiler will check that your code in the body of the method does not attempt to change any final parameters. Since the pass-by-value mechanism makes copies of values of the basic types, `final` really only makes sense when it is applied to parameters that are references to class objects, as we will see later on.

Specifying a parameter of a class as `final` is of limited value. It does prevent accidental modification of the object reference that is passed to the method, but it does not prevent modification of the object itself.

Defining Class Methods

You define a class method by adding the keyword `static` to its definition. For example, the class `Sphere` could have a class method to return the value stored in the static variable, `count`:

```

class Sphere {
    // Class definition as before...
}

```

```
// Static method to report the number of objects created
static int getCount() {
    return count; // Return current object count
}
}
```

This method needs to be a class method because we want to be able to get at the count of the number of objects even when it is zero. You can amend the `Sphere.java` file to include the definition of `getCount()`.

Note that you cannot directly refer to any of the instance variables in the class within a `static` method. This is because your `static` method may be executed when no objects of the class have been created, and therefore no instance variables exist.

Accessing Class Data Members in a Method

An instance method can access any of the data members of the class, just by using the appropriate name. Let's extend the class `Sphere` a little further by adding a method to calculate the volume of a `Sphere` object:

```
class Sphere {
    static final double PI = 3.14; // Class variable that has a fixed value
    static int count = 0; // Class variable to count objects

    // Instance variables
    double radius; // Radius of a sphere

    double xCenter; // 3D coordinates
    double yCenter; // of the center
    double zCenter; // of a sphere

    // Static method to report the number of objects created
    static int getCount(){
        return count; // Return current object count
    }

    // Instance method to calculate volume
    double volume() {
        return 4.0/3.0*PI*radius*radius*radius;
    }

    // Plus the rest of the class definition...
}
```

You can see that the method `volume()` is an instance method because it is not declared as `static`. It has no parameters but it does return a value of type `double` – the required volume. The method uses the class variable `PI` and the instance variable `radius` in the volume calculation – this is the expression $4.0/3.0 \cdot \text{PI} \cdot \text{radius} \cdot \text{radius} \cdot \text{radius}$ ($\frac{4}{3}\pi r^3$) in the return statement. The value that results from this expression will be returned to the point where the method is called for a `Sphere` object.

We know that each object of the class will have its own separate set of instance variables, so how is an instance variable for a particular object selected in a method? How does our `volume()` method pick up the radius for a particular `Sphere` object?

The Variable `this`

Every instance method has a variable with the name, `this`, which refers to the current object for which the method is being called. The compiler uses `this` implicitly when your method refers to an instance variable of the class. For example, when the method `volume()` refers to the instance variable `radius`, the compiler will insert the `this` object reference, so that the reference will be equivalent to `this.radius`. The return statement in the definition of the `volume()` method is actually:

```
return 4.0/3.0*PI*this.radius*this.radius*this.radius;
```

In general, every reference to an instance variable is in reality prefixed with `this`. You could put it in yourself, but there's no need, the compiler does it for you. In fact, it is not good practice to clutter up your code with `this` unnecessarily. However, there are occasions where you have to include it, as we shall see.

When you execute a statement such as:

```
double ballVolume = ball.volume();
```

where `ball` is an object of the class `Sphere`, the variable `this` in the method `volume()` will refer to the object `ball`, so the instance variable `radius` for this particular object will be used in the calculation.

We mentioned earlier that only one copy of each instance method for a class exists in memory, even though there may be many different objects. You can see that the variable `this` allows the same instance method to work for different class objects. Each time an instance method is called, the `this` variable is set to reference the particular class object to which it is being applied. The code in the method will then relate to the specific data members of the object referred to by `this`.

We have seen that there are four different potential sources of data available to you when you write the code for a method:

- ☐ Arguments passed to the method, which you refer to by using the parameter names.
- ☐ Data members, both instance variables and class variables, which you refer to by their variable names.
- ☐ Local variables declared in the body of the method.
- ☐ Values that are returned by other methods that are called from within the method.

The names of variables that are declared within a method are local to the method. You can use a name for a local variable or a parameter in a method that is the same as that of a class data member. If you find it necessary to do this then you must use the name `this` when you refer to the data member of the class from within the method. The variable name by itself will always refer to the variable that is local to the method, not the instance variable.

For example, let us suppose we wanted to add a method to change the radius of a `Sphere` object to a new radius value that is passed as an argument. We could code this as:

```
void changeRadius(double radius) {
    // Change the instance variable to the argument value
    this.radius = radius;
}
```

In the body of the `changeRadius()` method, `this.radius` refers to the instance variable, and `radius` by itself refers to the parameter. There is no confusion in the duplication of names here. It is clear that we are receiving a radius value as a parameter and storing it in the `radius` variable for the class object.

Initializing Data Members

We have seen how we were able to supply an initial value for the static members `PI` and `count` in the `Sphere` class with the declaration:

```
class Sphere {
    static final double PI = 3.14;           // Class variable that has a fixed value
    static int count = 0;                   // Class variable to count objects

    // Rest of the class...
}
```

We can also initialize ordinary non-static data members in the same way. For example:

```
class Sphere {
    static final double PI = 3.14;           // Class variable that has a fixed value
    static int count = 0;                   // Class variable to count objects

    // Instance variables
    double radius = 5.0;                    // Radius of a sphere

    double xCenter = 10.0;                  // 3D coordinates
    double yCenter = 10.0;                  // of the center
    double zCenter = 10.0;                  // of a sphere

    // Rest of the class...
}
```

Now every object of type `Sphere` will start out with a radius of 5.0 and have the center at the point 10.0, 10.0, 10.0.

There are some things that can't be initialized with a single expression. If you had a large array as a data member for example, that you wanted to initialize, with a range of values that required some kind of calculation, this would be a job for an **initialization block**.

Using Initialization Blocks

An initialization block is a block of code between braces that is executed before an object of the class is created. There are two kinds of initialization blocks. A **static initialization block** is a block defined using the keyword, `static`, and that is executed once when the class is loaded and can only initialize static data members of the class. A **non-static initialization block** is executed for each object that is created and thus can initialize instance variables in a class. This is easiest to understand by considering specific code.

Try It Out – Using an Initialization Block

Let's define a simple class with a static initialization block first of all:

```
class TryInitialization {
    static int[] values = new int[10];           // Static array member

    // Initialization block
    static {
        System.out.println("Running initialization block.");
        for(int i=0; i<values.length; i++)
            values[i] = (int)(100.0*Math.random());
    }

    // List values in the array for an object
    void listValues() {
        System.out.println();                   // Start a new line
        for(int i=0; i<values.length; i++)
            System.out.print(" " + values[i]);   // Display values

        System.out.println();                   // Start a new line
    }

    public static void main(String[] args) {
        TryInitialization example = new TryInitialization();
        System.out.println("\nFirst object:");
        example.listValues();

        example = new TryInitialization();
        System.out.println("\nSecond object:");
        example.listValues();
    }
}
```

When you compile and run this you will get identical sets of values for the two objects – as might be expected since the values array is static:

```
Running initialization block.
First object:
40 97 88 63 58 48 84 5 32 67

Second object:
40 97 88 63 58 48 84 5 32 67
```

How It Works

The `TryInitialization` class has a static member, `values`, that is an array of 10 integers. The static initialization block is the code:

```
static {
    System.out.println("Running initialization block.");
    for(int i=0; i<values.length; i++)
        values[i] = (int)(100.0*Math.random());
}
```

This initializes the `values` array with pseudo-random integer values generated in the `for` loop. The output statement in the block is there just to record when the initialization block executes. Because this initialization block is static, it is only ever executed once during program execution, when the class is loaded.

The `listValues()` method provides us with a means of outputting the values in the array. The `print()` method we are using in the `listValues()` method works just like `println()`, but without starting a new line before displaying the output, so we get all the values on the same line.

In `main()`, we generate an object of type `TryInitialization`, and then call its `listValues()` method. We then create a second object and call the `listValues()` method for that. The output demonstrates that the initialization block only executes once, and that the values reported for both objects are the same.

If you delete the modifier `static` from before the initialization block, and recompile and run the program again, you will get the output along the lines of:

```
Running initialization block.

First object:

66 17 98 59 99 18 40 96 40 21

Running initialization block.

Second object:

57 86 79 31 75 99 51 5 31 44
```

Now we have a non-static initialization block. You can see from the output that the values are different for the second object because the non-static initialization block is executed each time an object is created. In fact, the `values` array is static, so the array is shared between all objects of the class. You could demonstrate this by amending `main()` to store each object separately, and calling `listValues()` for the first object after the second object has been created. Amend the `main()` method in the program to read as follows:

```
public static void main(String[] args) {
    TryInitialization example = new TryInitialization();
    System.out.println("\nFirst object:");
    example.listValues();
}
```

```
TryInitialization nextexample = new TryInitialization();
System.out.println("\nSecond object:");
nextexample.listValues();

example.listValues();
}
```

While we have demonstrated that this is possible, you will not normally want to initialize static variables with a non-static initialization block.

As we said at the outset, a non-static initialization block can initialize instance variables too. If you want to demonstrate this too, you just need to remove the `static` modifier from the declaration of values and compile and run the program once more.

You can have multiple initialization blocks in a class, in which case they execute in the sequence in which they appear. The static blocks execute when the class is loaded and the non-static blocks execute when each object is created. Initialization blocks are useful, but you need more than that to create objects properly.

Constructors

When you create an object of a class, a special kind of method called a **constructor** is always invoked. If you don't define any constructors for your class, the compiler will supply a **default constructor** in the class that does nothing. The primary purpose of a constructor is to provide you with the means of initializing the instance variables uniquely for the object that is being created. If you are creating a `Person` object with the name John Doe, then you want to be able to initialize the member holding the person's name to "John Doe". This is precisely what a constructor can do. Any initialization blocks that you have defined in a class are always executed before a constructor.

A constructor has two special characteristics that differentiate it from other class methods:

- ❑ A constructor never returns a value and you must not specify a return type – not even of type `void`.
- ❑ A constructor always has the same name as the class.

To see a practical example we could add a constructor to our `Sphere` class definition:

```
class Sphere {
    static final double PI = 3.14; // Class variable that has a fixed value
    static int count = 0;          // Class variable to count objects

    // Instance variables
    double radius;                 // Radius of a sphere

    double xCenter;               // 3D coordinates
    double yCenter;               // of the center
    double zCenter;               // of a sphere
}
```

```

// Class constructor
Sphere(double theRadius, double x, double y, double z) {
    radius = theRadius;           // Set the radius

    // Set the coordinates of the center
    xCenter = x;
    yCenter = y;
    zCenter = z;
    ++count;                      // Update object count
}

// Static method to report the number of objects created
static int getCount() {
    return count;                // Return current object count
}

// Instance method to calculate volume
double volume() {
    return 4.0/3.0*PI*radius*radius*radius;
}
}

```

The definition of the constructor is shaded above. We are accumulating quite a lot of code to define the `Sphere` class, but as it's just an assembly of the pieces we have been adding, you should find it all quite straightforward.

As you can see, the constructor has the same name as the class and has no return type specified. A constructor can have any number of parameters, including none. The default constructor has no parameters. In our case we have four parameters, and each of the instance variables is initialized with the value of the appropriate parameter. Here is a situation where we might have used the name `radius` for the parameter, in which case we would need to use the keyword `this` to refer to the instance variable of the same name. The last action of our constructor is to increment the class variable, `count`, by 1, so that `count` accumulates the total number of objects created.

The Default Constructor

If you don't define any constructors for a class, the compiler will supply a default constructor that has no parameters and does nothing. Before we defined a constructor for our `Sphere` class, the compiler would have supplied one defined like this:

```

Sphere() {
}

```

It has no parameters and has no statements in its body so it does nothing – except enable you to create an object of type `Sphere` of course. The object created by the default constructor will have fields with their default values set.

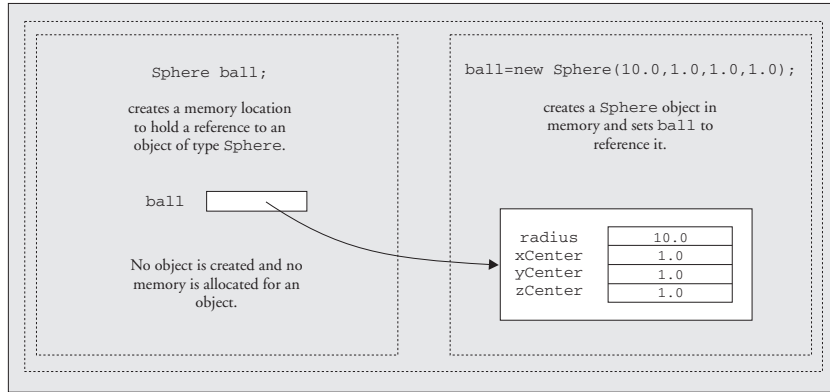
Note that if you define a constructor of any kind for a class, the compiler will not supply a default constructor. If you need one – and there are occasions when you do – you must define it explicitly.

Creating Objects of a Class

When you declare a variable of type `Sphere` with the statement:

```
Sphere ball;    // Declare a variable
```

no constructor is called and no object is created. All you have created at this point is the variable `ball`, which can store a reference to an object of type `Sphere`, if and when you create one.



You will recall from our discussion of `String` objects and arrays that the variable and the object it references are distinct entities. To create an object of a class you must use the keyword `new` followed by a call to a constructor. To initialize `ball` with an object, you could write:

```
ball = new Sphere(10.0, 1.0, 1.0, 1.0);    // Create a sphere
```

Now we have a `Sphere` object with a radius of 10.0 located at the coordinates (1.0, 1.0, 1.0). The object is created in memory and will occupy a sufficient number of bytes to accommodate all the data necessary to define the object. The variable `ball` will record where in memory the object is – it acts as a reference to the object.

Of course, you can do the whole thing in one step, with the statement:

```
Sphere ball = new Sphere(10.0, 1.0, 1.0, 1.0);    // Create a sphere
```

This declares the variable `ball` and defines the `Sphere` object to which it refers.

You can create another variable that refers to the same object as `ball`:

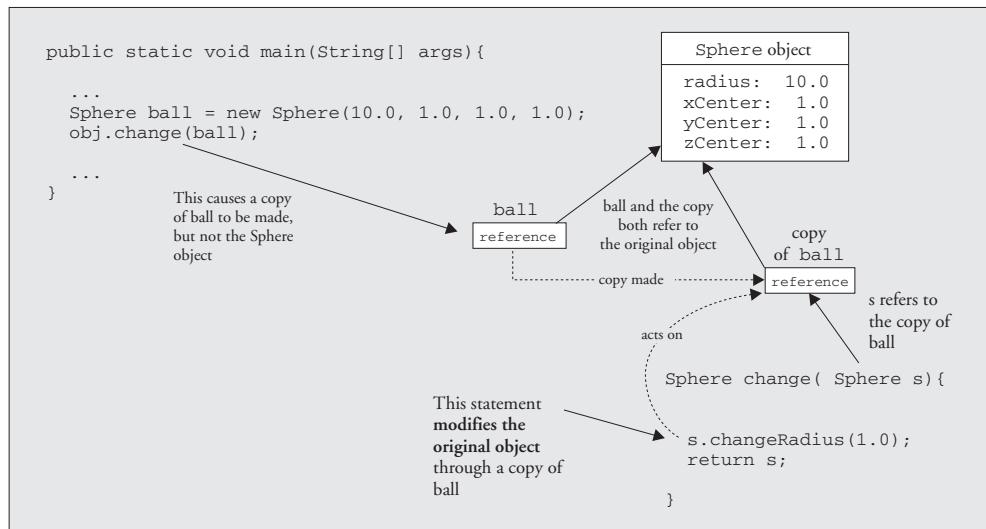
```
Sphere myBall = ball;
```

Now the variable `myBall` refers to the same object as `ball`. We have only one object still, but we have two different variables that reference it. You could have as many variables as you like referring to the same object.

The separation of the variable and the object has an important effect on how objects are passed to a method, so we need to look at that.

Passing Objects to a Method

When you pass an object as an argument to a method, the mechanism that applies is called **pass-by-reference**, because a copy of the reference contained in the variable is transferred to the method, not the object itself. The effect of this is shown in the following diagram.



This illustration presumes we have defined a method, `changeRadius()`, in the class `Sphere` that will alter the radius value for an object, and that we have a method `change()` in some other class that calls `changeRadius()`. When the variable `ball` is used as an argument to the method `change()`, the pass-by-reference mechanism causes a copy of `ball` to be made and stored in `s`. The variable `ball` just stores a reference to the `Sphere` object, and the copy contains that same reference and therefore refers to the same object. No copying of the actual object occurs. This is a major plus in terms of efficiency when passing arguments to a method. Objects can be very complex involving a lot of instance variables. If objects themselves were always copied when passed as arguments, it could be very time consuming and make the code very slow.

Since the copy of `ball` refers to the same object as the original, when the `changeRadius()` method is called the original object will be changed. You need to keep this in mind when writing methods that have objects as parameters because this is not always what you want.

In the example shown, the method `change()` returns the modified object. In practice you would probably want this to be a distinct object, in which case you would need to create a new object from `s`. You will see how you can write a constructor to do this a little later in this chapter.

Remember that this only applies to objects. If you pass a variable of type `int` or `double` to a method for example, a copy of the value is passed. You can modify the value passed as much as you want in the method, but it won't affect the original value.

The Lifetime of an Object

The lifetime of an object is determined by the variable that holds the reference to it – assuming there is only one. If we have the declaration:

```
Sphere ball = new Sphere(10.0, 1.0, 1.0, 1.0); // Create a sphere
```

then the `Sphere` object that the variable `ball` refers to will die when the variable `ball` goes out of scope. This will be at the end of the block containing this declaration. Where an instance variable is the only one referencing an object, the object survives as long as the instance variable owning the object survives.

A slight complication can arise with objects though. As you have seen, several variables can reference a single object. In this case the object survives as long as there is still a variable in existence somewhere that references the object.

You can reset a variable to refer to nothing by setting its value to `null`. If you write the statement:

```
ball = null;
```

the variable `ball` no longer refers to an object, and assuming there is no other object referencing it, the `Sphere` object it originally referenced will be destroyed. Note that while the object has been discarded, the variable `ball` still continues to exist. The lifetime of the object is determined by whether any variable anywhere in the program still references it.

The process of disposing of dead objects is called **garbage collection**. Garbage collection is automatic in Java, but this doesn't necessarily mean that objects disappear from memory straight away. It can be some time after the object becomes inaccessible to your program. This won't affect your program directly in any way. It just means you can't rely on memory occupied by an object that is done with being available immediately. For the most part it doesn't matter; the only circumstances where it might would be if your objects were very large, millions of bytes say, or you were creating and getting rid of very large numbers of objects. In this case you can call the static `gc()` method defined in the `System` class to encourage the JVM to do some garbage collecting and recover the memory that the objects occupy:

```
System.gc();
```

This is a best efforts deal on the part of the JVM. When the `gc()` method returns, the JVM will have tried to reclaim the space occupied by discarded objects, but there's no guarantee that it will all be recovered.

Defining and Using a Class

To put what we know about classes to use, we can use our `Sphere` class in an example.

You will be creating two source files. The first is the file `CreateSpheres.java`, which will contain the definition of the `CreateSpheres` class that will have the method `main()` defined as a static method. As usual, this is where execution of the program starts. The second file will be the file `Sphere.java` that contains the definition of the class `Sphere` that we have been assembling.

Both files will need to be in the same directory or folder – I suggest you name the directory `CreateSpheres`. Then copy or move the last version of `Sphere.java` to this directory.

Try It Out – Using the Sphere Class

Enter the following code for the file `CreateSpheres.java`:

```
class CreateSpheres {
    public static void main(String[] args) {
        System.out.println("Number of objects = " + Sphere.getCount());

        Sphere ball = new Sphere(4.0, 0.0, 0.0, 0.0);        // Create a sphere
        System.out.println("Number of objects = " + ball.getCount());

        Sphere globe = new Sphere(12.0, 1.0, 1.0, 1.0);      // Create a sphere
        System.out.println("Number of objects = " + Sphere.getCount());

        // Output the volume of each sphere
        System.out.println("ball volume = " + ball.volume());
        System.out.println("globe volume = " + globe.volume());
    }
}
```

Compile the source files and then run `CreateSpheres`, and you should get the output:

```
Number of objects = 0
Number of objects = 1
Number of objects = 2
ball volume = 267.94666666666666
globe volume = 7234.5599999999999
```

This is the first time we have run a program involving two source files. If you are using the JDK compiler, then compile `CreateSpheres.java` with the current directory as `CreateSpheres` using the command:

```
javac CreateSpheres.java
```

The compiler will find and compile the `Sphere.java` source file automatically. If all the source files for a program are in the current directory, then compiling the file containing a definition of `main()` will compile all the source files for the program.

Note that by default, the `.class` files generated by the compiler will be stored in the current directory, that is, the directory containing your sourcecode. If you want the `.class` files stored in a different directory, then you can use the `-d` option with the Java compiler to specify where they should go. For example, to store the class files in a directory called `C:\classes` you would type:

```
javac -d C:/classes CreateSpheres.java
```

How It Works

The `Sphere` class definition includes a constructor and the method `volume()` to calculate the volume of a particular sphere. It also contains the static method, `getCount()`, we saw earlier, which returns the current value of the class variable `count`. We need to define this method as `static` since we want to be able to call it regardless of how many objects have been created, including the situation when there are none.

The method `main()` in the `CreateSpheres` class puts the class `Sphere` through its paces. When the program is compiled, the compiler will look for a file `Sphere.java` to provide the definition of the class `Sphere`. As long as this file is in the current directory the compiler will be able to find it.

The first thing the program does is to call the static method `getCount()`. Because no objects exist, you must use the class name to call it at this point. We then create the object `ball`, which is a `Sphere` object, with a radius of 4.0 and its center at the origin point, (0.0, 0.0, 0.0). The method `getCount()` is called again, this time using the object name to demonstrate that you can call a static method through an object. Another `Sphere` object, `globe`, is created with a radius of 12.0. The `getCount()` method is called again, this time using the class name. Static methods are usually called using the class name because in most situations, where you would use such a method, you cannot be sure that any objects exist. After all, the reason for calling this particular method would be to find out how many objects exist. A further reason to use the class name when calling a static method is that it makes it quite clear in the sourcecode that it is a static method that is being called. You can't call a non-static method using the class name.

Our program finally outputs the volume of both objects by calling the `volume()` method for each, from within the expressions, specifying the arguments to the `println()` method calls.

Method Overloading

Java allows you to define several methods in a class with the same name, as long as each method has a set of parameters that is unique. This is called **method overloading**.

The name of a method together with the type and sequence of the parameters form the **signature** of the method – the signature of each method in a class must be distinct to allow the compiler to determine exactly which method you are calling at any particular point.

Note that the return type has no effect on the signature of a method. You cannot differentiate between two methods just by the return type. This is because the return type is not necessarily apparent when you call a method. For example, suppose you write a statement such as:

```
Math.round(value);
```

Although the statement above is pointless since we discard the value that the `round()` method produces, it does illustrate why the return type cannot be part of the signature for a method. There is no way for the compiler to know from this statement what the return type of the method `round()` is supposed to be. Thus, if there were several different versions of the method `round()`, and the return type was the only distinguishing aspect of the method signature, the compiler would be unable to determine which version of `round()` you wanted to use.

There are many circumstances where it is convenient to use method overloading. You have already seen that the standard class `Math` contains two versions of the method `round()`, one that accepts an argument of type `float`, and the other that accepts an argument of type `double`. You can see now that method overloading makes this possible. It would be rather tedious to have to use a different name for each version of `round()` when they both do essentially the same thing. The `valueOf()` method in the `String` class is another example. There is a version of this method for each of the basic types. One context in which you will regularly need to use overloading is when you write constructors for your classes, which we'll look at now.

Multiple Constructors

Constructors are methods that can be overloaded, just like any other method in a class. In most situations, you will need to generate objects of a class from different sets of initial defining data. If we just consider our class `Sphere`, we could conceive of a need to define a `Sphere` object in a variety of ways. You might well want a constructor that accepted just the (x, y, z) coordinates of a point, and have a `Sphere` object created with a default radius of 1.0. Another possibility is that you may want to create a default `Sphere` with a radius of 1.0 positioned at the origin, so no arguments would be specified at all. This requires two constructors in addition to the one we have already written.

Try It Out – Multiple Constructors for the Sphere Class

The code for the extra constructors is:

```
class Sphere {
    // First Constructor and variable declarations
    ...
    // Construct a unit sphere at a point
    Sphere(double x, double y, double z) {
        xCenter = x;
        yCenter = y;
        zCenter = z;
        radius = 1.0;
        ++count;                // Update object count
    }

    // Construct a unit sphere at the origin
    Sphere() {
        xCenter = 0.0;
        yCenter = 0.0;
        zCenter = 0.0;
        radius = 1.0;
        ++count;                // Update object count
    }

    // The rest of the class as before...
}
```

The statements in the default constructor that set three data members to zero are not really necessary, as the data members would be set to zero by default. They are there just to emphasize that the primary purpose of a constructor is to enable you to set initial values for the data members.

If you add the following statements to the `CreateSpheres` class, you can test out the new constructors:

```
public class CreateSpheres {
    public static void main(String[] args) {
        System.out.println("Number of objects = " + Sphere.getCount());

        Sphere ball = new Sphere(4.0, 0.0, 0.0, 0.0);           // Create a sphere
        System.out.println("Number of objects = " + ball.getCount());

        Sphere globe = new Sphere(12.0, 1.0, 1.0, 1.0);         // Create a sphere
        System.out.println("Number of objects = " + Sphere.getCount());

        Sphere eightBall = new Sphere(10.0, 10.0, 0.0);
        Sphere oddBall = new Sphere();
        System.out.println("Number of objects = " + Sphere.getCount());

        // Output the volume of each sphere
        System.out.println("ball volume = " + ball.volume());
        System.out.println("globe volume = " + globe.volume());
        System.out.println("eightBall volume = " + eightBall.volume());
        System.out.println("oddBall volume = " + oddBall.volume());
    }
}
```

Now the program should produce the output:

```
Number of objects = 0
Number of objects = 1
Number of objects = 2
Number of objects = 4
ball volume = 267.94666666666666
globe volume = 7234.5599999999999
eightBall volume = 4.1866666666666665
oddBall volume = 4.1866666666666665
```

How It Works

When you create a `Sphere` object, the compiler will select the constructor to use based on the types of the arguments you have specified. So, the first of the new constructors is applied in the first statement that we added to `main()`, as its signature fits with the argument types used. The second statement that we added clearly selects the last constructor as no arguments are specified. The other additional statements are there just to generate some output corresponding to the new objects. You can see from the volumes of `eightBall` and `oddBall` that they both are of radius 1.

It is the number and types of the parameters that affect the signature of a method, not the parameter names. If you wanted a constructor that defined a `Sphere` object at a point, by specifying the diameter rather than the radius, you have a problem. You might try to write it as:

```
// Illegal constructor!!!
// This WON'T WORK because it has the same signature as the original!!!
Sphere(double diameter, double x, double y, double z) {
    xCenter = x;
    yCenter = y;
    zCenter = z;
    radius = diameter/2.0;
}
```

If you try adding this to the `Sphere` class and recompile, you'll get a compile-time error. This constructor has four arguments of type `double`, so its signature is identical to the first constructor that we wrote for the class. This is not permitted – hence the compile-time error. When the number of parameters is the same in two overloaded methods, at least one pair of corresponding parameters must be of different types.

Calling a Constructor from a Constructor

One class constructor can call another constructor in the same class in its first executable statement. This can often save duplicating a lot of code. To refer to another constructor in the same class, you use `this` as the name, followed by the appropriate arguments between parentheses. In our `Sphere` class, we could have defined the constructors as:

```
class Sphere {
    // Construct a unit sphere at the origin
    Sphere() {
        radius = 1.0;
        // Other data members will be zero by default
        ++count;                // Update object count
    }

    // Construct a unit sphere at a point
    Sphere(double x, double y, double z)
    {
        this();                // Call the constructor with no arguments
        xCenter = x;
        yCenter = y;
        zCenter = z;
    }

    Sphere(double theRadius, double x, double y, double z) {
        this(x, y, z);        // Call the 3 argument constructor
        radius = theRadius;    // Set the radius
    }

    // The rest of the class as before...
}
```

In the constructor that accepts the point coordinates as argument, we call the default constructor to set the radius and increment the count of the number of objects. In the constructor that sets the radius, as well as the coordinates, the constructor with three arguments is called to set the coordinates, which in turn will call the constructor that requires no arguments.

Duplicating Objects Using a Constructor

When we were looking at how objects were passed to a method, we came up with a requirement for duplicating an object. The need to produce an identical copy of an object occurs surprisingly often.

Java provides a `clone()` method, but the details of using it must wait for the next chapter.

Suppose you declare a `Sphere` object with the following statement:

```
Sphere eightBall = new Sphere(10.0, 10.0, 0.0);
```

Later in your program you want to create a new object `newBall`, which is identical to the object `eightBall`. If you write:

```
Sphere newBall = eightBall;
```

this will compile OK but it won't do what you want. You will remember from our earlier discussion that the variable `newBall` will reference the same object as `eightBall`. You will not have a distinct object. The variable `newBall`, of type `Sphere`, is created but no constructor is called, so no new object is created.

Of course, you could create `newBall` by specifying the same arguments to the constructor as you used to create `eightBall`. In general, however, it may be that `eightBall` has been modified in some way during execution of the program, so you don't know that its instance variables have the same values – for example, the position might have changed. This presumes that we have some other class methods that alter the instance variables. You could fix this by adding a constructor to the class that will accept an existing `Sphere` object as an argument:

```
// Create a sphere from an existing object
Sphere(final Sphere oldSphere) {
    radius = oldSphere.radius;
    xCenter = oldSphere.xCenter;
    yCenter = oldSphere.yCenter;
    zCenter = oldSphere.yCenter;
}
```

This works by copying the values of the instance variables of the `Sphere` object, passed as an argument, to the corresponding instance variables of the new object.

Now you can create `newBall` as a distinct object by writing:

```
Sphere newBall = new Sphere(eightBall); // Create a copy of eightBall
```

Let's recap what we have learned about methods and constructors with another example.

Using Objects

Let's create an example to do some simple 2D geometry. This will give us an opportunity to use more than one class. We will define two classes, a class of point objects and a class of line objects – we then use these to find the point at which the lines intersect. We will call the example `TryGeometry`, so this will be the name of the directory or folder in which you should save the program files. Quite a few lines of code are involved so we will put it together piecemeal, and try to understand how each piece works as we go.

Try It Out – The Point Class

We first define a basic class for point objects:

```
class Point {
    // Coordinates of the point
    double x;
    double y;

    // Create a point from coordinates
    Point(double xVal, double yVal) {
        x = xVal;
        y = yVal;
    }

    // Create a point from another Point object
    Point(final Point oldPoint) {
        x = oldPoint.x;    // Copy x coordinate
        y = oldPoint.y;    // Copy y coordinate
    }

    // Move a point
    void move(double xDelta, double yDelta) {
        // Parameter values are increments to the current coordinates
        x += xDelta;
        y += yDelta;
    }

    // Calculate the distance to another point
    double distance(final Point aPoint) {
        return Math.sqrt(
            (x - aPoint.x)*(x - aPoint.x) + (y - aPoint.y)*(y - aPoint.y) );
    }

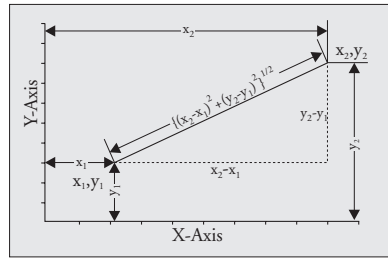
    // Convert a point to a string
    public String toString() {
        return Double.toString(x) + ", " + y;    // As "x, y"
    }
}
```

You should save this as `Point.java` in the directory `TryGeometry`.

How It Works

This is a simple class that has just two instance variables, `x` and `y`, which are the coordinates of the `Point` object. At the moment we have two constructors. One will create a point from a coordinate pair passed as arguments, and the other will create a new `Point` object from an existing one.

There are three methods included in the class. First we have the `move()` method that moves a `Point` to another position by adding an increment to each of the coordinates. We also have the `distance()` method that calculates the distance from the current `Point` object to the `Point` object passed as an argument. This uses the Pythagorean theorem to compute the distance as shown below.



Finally we have a method `toString()` that returns a string representation of the coordinates of the current point. If a class defines the `toString()` method, an object of that class can be used as an operand of the string concatenation operator `+`, so you can implement this in any of your classes to allow objects to be used in this way. The compiler will automatically insert a call to `toString()` when necessary. For example, suppose `thePoint` is an object of type `Point`, and we write the statement:

```
System.out.println("The point is at " + thePoint);
```

The `toString()` method will be automatically invoked to convert `thePoint` to a `String`, and the result will be appended to the `String` literal. We have specified the `toString()` method as `public`, as this is essential here for the class to compile. We will defer explanations as to why this is so until later in this chapter.

Note how we use the static `toString()` method defined in the class `Double` to convert the `x` value to a `String`. The compiler will insert a call to the same method automatically for the `y` value as the left operand of the `+` operation is a `String` object. Note that we could equally well have used the `valueOf()` method in the `String` class. In this case the statement would be written like this:

```
return String.valueOf(x) + ", " + y;    // As "x, y"
```

Try It Out – The Line Class

We can use `Point` objects in the definition of the class `Line`:

```
class Line {
    Point start;    // Start point of line
    Point end;      // End point of line

    // Create a line from two points
    Line(final Point start, final Point end) {
        this.start = new Point(start);
        this.end = new Point(end);
    }

    // Create a line from two coordinate pairs
    Line(double xStart, double yStart, double xEnd, double yEnd) {
        start = new Point(xStart, yStart);    // Create the start point
        end = new Point(xEnd, yEnd);          // Create the end point
    }
}
```

```

// Calculate the length of a line
double length() {
    return start.distance(end); // Use the method from the Point class
}

// Convert a line to a string
public String toString() {
    return "(" + start + "):(" + end + ")"; // As "(start):(end)"
                                           // that is, "(x1, y1):(x2, y2)"
}

```

You should save this as `Line.java` in the directory `TryGeometry`.

How It Works

You shouldn't have any difficulty with this class definition, as it is very straightforward. The class `Line` stores two `Point` objects as instance variables. There are two constructors for `Line` objects, one accepting two `Point` objects as arguments, the other accepting the (x, y) coordinates of the start and end points. You can see how we use the variable `this` to differentiate the class instance variables, `start` and `end`, from the parameter names in the constructor.

Note how the constructor that accepts `Point` objects works:

```

// Create a line from two points
Line(final Point start, final Point end) {
    this.start = new Point(start);
    this.end = new Point(end);
}

```

With this implementation of the constructor, two new `Point` objects are created which will be identical to, but independent of, the objects passed to the constructor. If you don't think about what happens you might be tempted to write it as:

```

// Create a line from two points
Line(final Point start, final Point end) {
    this.start = start; // Dependent on external object!!!
    this.end = end; // Dependent on external object!!!
}

```

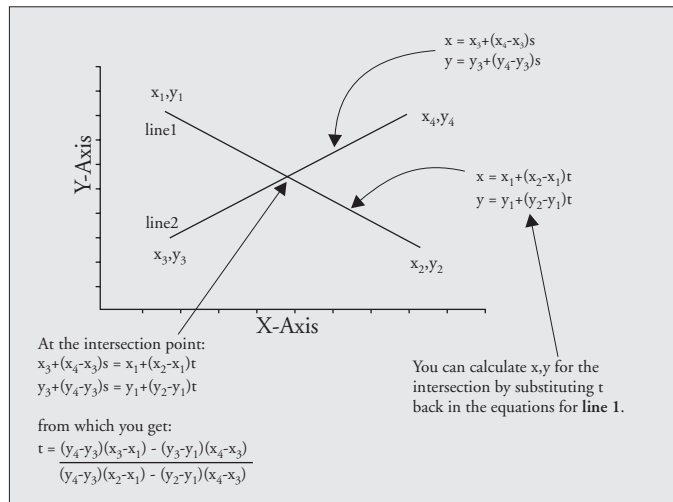
The important thing you should notice here, is that the way the constructor is implemented could cause problems that might be hard to track down. It's the same problem of an object variable being separate from the object to which it refers. In this version of the constructor no new points are created. The `start` and `end` members of the object refer to the `Point` objects passed as arguments. The `Line` object will be implicitly dependent on the `Point` objects that are used to define it. If these were changed outside the `Line` class, by using the `move()` method for example, this would 'silently' modify the `Line` object. You might consciously decide that this is what you want, so the `Line` object continues to be dependent on its associated `Point` objects, for instance in a drawing package. But, in general, you should avoid implicit dependencies between objects.

In the `toString()` method for the `Line` class, we are able to use the `Point` objects directly in the formation of the `String` representation of a `Line` object. This works because the `Point` class also defines a `toString()` method.

We've now defined two classes. In these class definitions, we've included the basic data that defines an object of each class. We've also defined some methods which we think will be useful, and added constructors for a variety of input parameters. Note how the `Point` class is used in the definition of the `Line` class. It is quite natural to define a line in terms of two `Point` objects, and the `Line` class is much simpler and more understandable than if it was defined entirely in terms of the individual `x` and `y` coordinates. To further demonstrate how classes can interact, and how you can solve problems directly, in terms of the objects involved, let's devise a method to calculate the intersection of two `Line` objects.

Creating a Point from Two Lines

We could add this method to the `Line` class. The diagram below illustrates how the mathematics works out.



You can ignore the mathematics if you want to, as it is not the most important aspect of the example. If you are willing to take the code in the new constructor on trust, then skip to the *Try It Out* section below. On the other hand you shouldn't find it too difficult if you can still remember what you did in high school.

One way to get the intersection of two lines is to use equations like those shown. These are called parametric equations because they use a parameter value (s or t) as the variable for determining points on each line. The parameters s and t vary between 0 and 1 to give points on the lines between the defined start and end points. When a parameter s or t is 0 the equations give the coordinates of the start point of each line, and when the parameter value is 1 you get the end point of the line.

Where the lines intersect, the equations for the lines must produce the same (x, y) values, so, at this point, the right-hand sides of the equations for x for the two lines must be equal, and the same goes for the equations for y . This will give you two equations in s and t , and with a bit of algebraic juggling you can eliminate s to get the equation shown for t . You can then replace t in the equations defining line 1 to get x and y for the intersection point.

Try It Out – Calculating the Intersection of Two Lines

We can use these results to write the additional method we need for the `Line` class. Add the following code to the definition in `Line.java`:

```
// Return a point as the intersection of two lines -- called from a Line object
Point intersects(final Line line1) {
    Point localPoint = new Point(0, 0);

    double num = (this.end.y - this.start.y)*(this.start.x - line1.start.x) -
        (this.end.x - this.start.x)*(this.start.y - line1.start.y);

    double denom = (this.end.y - this.start.y)*(line1.end.x - line1.start.x) -
        (this.end.x - this.start.x)*(line1.end.y - line1.start.y);

    localPoint.x = line1.start.x + (line1.end.x - line1.start.x)*num/denom;
    localPoint.y = line1.start.y + (line1.end.y - line1.start.y)*num/denom;

    return localPoint;
}
```

Since the `Line` class definition refers to the `Point` class, the `Line` class can't be compiled without the other being available. When you compile the `Line` class the compiler will compile the other class too.

How It Works

The `intersects()` method is called from one `Line` object, and takes another `Line` object as an argument. In the code, the local variables `num` and `denom` are the numerator and denominator in the expression for t in the diagram. We then use these values to calculate the x and y coordinates for the intersection.

If the lines are parallel, the denominator in the equation for t will be zero, something you should really check for in the code. For the moment, we will ignore it and end up with coordinates that are *Infinity* if it occurs.

Note how we get at the values of the coordinates for the `Point` objects defining the lines. The dot notation for referring to a member of an object is just repeated when you want to reference a member of a member. For example, for the object `line1`, the expression `line1.start` refers to the `Point` object at the beginning of the line. Therefore `line1.start.x` refers to its x coordinate, and `line1.start.y` accesses its y coordinate.

Now we have a `Line` class, which we can use to calculate the intersection point of two `Line` objects. We need a program to test the code out.

Try It Out – The TryGeometry Class

We can demonstrate the two classes we have defined, with the following code in the method `main()`:

```
public class TryGeometry {
    public static void main(String[] args) {
        // Create two points and display them
        Point start = new Point(0.0, 1.0);
```

```
Point end = new Point(5.0, 6.0);
System.out.println("Points created are " + start + " and " + end);

// Create two lines and display them
Line line1 = new Line(start, end);
Line line2 = new Line(0.0, 3.0, 3.0, 0.0);
System.out.println("Lines created are " + line1 + " and " + line2);

// Display the intersection
System.out.println("Intersection is " + line2.intersects(line1));

// Now move the end point of line1 and show the new intersection
end.move(1.0, -5.0);
System.out.println("Intersection is " + line1.intersects(line2));
}
}
```

The program will produce the output:

```
Points created are 0.0, 1.0 and 5.0, 6.0
Lines created are (0.0, 1.0):(5.0, 6.0) and (0.0, 3.0):(3.0, 0.0)
Intersection is 1.0, 2.0
Intersection is 1.0, 2.0
```

How It Works

We first create two `Point` objects, which we will use later in the creation of the object `line1`. We then display the points using the `println()` method. The `toString()` method that we defined in the `Point` class is used automatically to generate the `String` representation for each `Point` object.

After creating `line1` from our two points, we use the other constructor in the `Line` class to create `line2` from two pairs of coordinates. We then display the two lines. The `toString()` member of the `Line` class is invoked here to create the `String` representation of each `Line` object, and this in turn uses the `toString()` method in the `Point` class.

The next statement calls the `intersects()` method from the `line2` object and returns the `Point` object at the intersection of the two lines, `line1` and `line2`, as part of the argument to the `println()` method that outputs the point. As you see, we are not obliged to save an object when we create it. Here we just use it to create the string to be displayed.

We use the `move()` method in the class `Point` to modify the coordinates of the object, `end`, that we used to create `line1`. We then get the intersection of the two lines again, this time calling the `intersects()` method from `line1`. The output result demonstrates that `line1` is independent of the object `end`, as moving the point has made no difference to the intersection.

If you change the constructor in the `Line` class, to the version we saw earlier that does not create new `Point` objects to define the line, you can run the example again to see the effect. The output will be:

```
Points created are 0.0, 1.0 and 5.0, 6.0
Lines created are (0.0, 1.0):(5.0, 6.0) and (0.0, 3.0):(3.0, 0.0)
Intersection is 1.0, 2.0
Intersection is 2.0, 1.0
```

Changing the end object now alters the line, so we get a different intersection point for the two lines after we move the point end. This is because the `Line` object, `line1`, contains references to the `Point` objects defined in `main()`, not independent `Point` objects.

Recursion

The methods you have seen so far have been called from within other methods, but a method can also call itself – something referred to as **recursion**. Clearly you must include some logic in a recursive method so that it will eventually stop calling itself. We can see how this might be done with a simple example.

We can write a method that will calculate integer powers of a variable, in other words, evaluate x^n , or $x * x * \dots * x$ where x is multiplied by itself n times. We can use the fact that we can obtain x^n by multiplying x^{n-1} by x . To put this in terms of a specific example, we can calculate 2^4 as 2^3 multiplied by 2, and we can get 2^3 by multiplying 2^2 by 2, and 2^2 is produced by multiplying 2^1 , which is 2 of course, by 2.

Try It Out – Calculating Powers

Here is the complete program including the recursive method, `power()`:

```
public class PowerCalc {
    public static void main(String[] args) {
        double x = 5.0;
        System.out.println(x + " to the power 4 is " + power(x,4));
        System.out.println("7.5 to the power 5 is " + power(7.5,5));
        System.out.println("7.5 to the power 0 is " + power(7.5,0));
        System.out.println("10 to the power -2 is " + power(10,-2));
    }

    // Raise x to the power n
    static double power(double x, int n) {
        if(n > 1)
            return x*power(x, n-1);    // Recursive call
        else if(n < 0)
            return 1.0/power(x, -n);  // Negative power of x
        else
            return n == 0 ? 1.0 : x;    // When n is 0 return 1, otherwise x
    }
}
```

This program will produce the output:

```
5.0 to the power 4 is 625.0
7.5 to the power 5 is 23730.46875
7.5 to the power 0 is 1.0
10 to the power -2 is 0.01
```

How It Works

The method `power()` has two parameters, the value `x` and the power `n`. The method performs four different actions, depending on the value of `n`:

$n > 1$	A recursive call to power() is made with n reduced by 1, and the value returned is multiplied by x . This is effectively calculating x^n as x times x^{n-1} .
$n < 0$	x^{-n} is equivalent to $1/x^n$ so this is the expression for the return value. This involves a recursive call to <code>power()</code> with the sign of n reversed.
$n = 0$	x^0 is defined as 1, so this is the value returned.
$n = 1$	x^1 is x , so x is returned.

Just to make sure the process is clear we can work through the sequence of events as they occur in the calculation of 5^4 .

Level	Description	Relevant Code
1	The first call of the <code>power()</code> method passes 5.0 and 4 as arguments. Since the second argument, n , is greater than 1, the <code>power()</code> method is called again in the return statement, with the second argument reduced by 1.	<pre>Power(5.0, 4) { if(n > 1) return 5.0*power(5.0, 4-1); ... }</pre>
2	The second call of the <code>power()</code> method passes 5.0 and 3 as arguments. Since the second argument, n , is still greater than 1, the <code>power()</code> method is called again in the return statement with the second argument reduced by 1.	<pre>Power(5.0, 3) { if(n > 1) return 5.0*power(5.0, 3-1); ... }</pre>
3	The third call of the <code>power()</code> method passes 5.0 and 2 as arguments. Since the second argument, n , is still greater than 1, the <code>power()</code> method is called again, with the second argument again reduced by 1.	<pre>Power(5.0, 2) { if(n > 1) return 5.0*power(5.0, 2-1); ... }</pre>
4	The fourth call of the <code>power()</code> method passes 5.0 and 1 as arguments. Since the second argument, n , is not greater than 1, the value of the first argument, 5.0, is returned to level 3.	<pre>Power(5.0, 1) { if(n > 1) ... else return 5.0; }</pre>

Level	Description	Relevant Code
3	Back at level 3, the value returned, 5.0, is multiplied by the first argument, 5.0, and returned to level 2.	<pre>Power(5.0, 2) { if(n>1) ... else return 5.0*5.0; }</pre>
2	Back at level 2, the value returned, 25.0, is multiplied by the first argument, 5.0, and returned to level 1.	<pre>Power(5.0, 3) { if(n > 1) ... else return 5.0*25.0; }</pre>
1	Back at level 1, the value returned, 125.0, is multiplied by the first argument, 5.0, and 625.0 is returned as the result of calling the method in the first instance.	<pre>Power(5.0, 4) { if(n > 1) ... else return 5.0*125.0; }</pre>

You can see from this that the method `power()` is called four times in all. The calls cascade down through four levels until the value of `n` is such that it allows a value to be returned. The return values ripple up through the levels until we are eventually back at the top, and 625.0 is returned to the original calling point.

As a rule, you should only use recursion where there are evident advantages in the approach, as there is quite a lot of overhead in recursive method calls. This particular example could be more easily programmed as a loop and it would execute much more efficiently. One example of where recursion can be applied very effectively is in the handling of data structures such as trees. Unfortunately these don't make convenient illustrations of how recursion works at this stage of the learning curve, because of their complexity.

Before we can dig deeper into classes, we need to take an apparent detour to understand what a package is in Java.

Understanding Packages

Packages are fundamental to Java programs so make sure you understand this section.

Packages are implicit in the organization of the standard classes as well as your own programs, and they influence the names you can use for classes and the variables and methods they contain. Essentially, a **package** is a unique named collection of classes. The purpose of grouping classes in a package with a unique name is to make it easy to add any or all of the classes in a package into your program code. One aspect of this is that the names used for classes in one package will not interfere with the names of classes in another package, or your program, because the class names in a package are qualified by the package name.

Every class in Java is contained in a package, including all those we have defined in our examples. You haven't seen many references to package names so far because we have been implicitly using the **default package** to hold our classes, and this doesn't have a name.

All of the standard classes in Java are contained within packages. The package that contains most of the standard classes that we have used so far is called `java.lang`. You haven't seen any explicit reference to this in your code either, because this package is automatically available to your programs. Things are arranged this way because some of the classes in `java.lang`, such as `String`, are used in every program. There are other packages containing standard classes that you will need to include explicitly when you use them, as we did in the previous chapter with the `StringTokenizer` class.

Packaging Up Your Classes

Putting one of your classes in a package is very simple. You just add a package statement as the first statement in the source file containing the class definition. Note that it must always be the first statement. Only comments or blank lines are allowed to precede the package statement. A **package statement** consists of the keyword, `package`, followed by the package name, and is terminated by a semi-colon. If you want the classes in a package to be accessible outside the package, you must declare the class using the keyword `public` in the first line of your class definition. Class definitions that aren't preceded by the keyword `public` are only accessible from methods in classes that belong to the same package.

For example, to include the class `Sphere` in a package called `Geometry`, the contents of the file `Sphere.java` would need to be:

```
package Geometry;

public class Sphere {
    // Details of the class definition
}
```

Each class that you want to include in the package `Geometry` must contain the same package statement at the beginning, and you should save all the files for the classes in the package in a directory with the same name as the package, that is, `Geometry`. Note the use of the `public` keyword in the definition of the `Sphere` class. This is to make the class usable generally.

Note that you would also need to declare the constructors and methods in the class as `public` if you want them to be accessible from outside of the package. We will return to this in more detail a little later in this chapter.

Packages and the Directory Structure

Packages are actually a little more complicated than they appear at first sight, because a package is intimately related to the directory structure in which it is stored. You already know that the definition of a class with the name `ClassName` must be stored in a file with the name `ClassName.java`, and that all the files for classes within a package, `PackageName`, must be included in a directory with the name `PackageName`. You can compile the source for a class within a package and have the `.class` file that is generated stored in a different directory, but the directory name must still be the same as the package name.

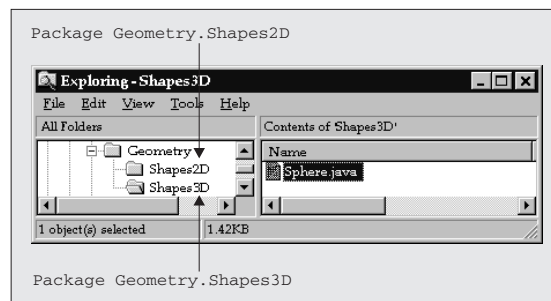
A package need not have a single name. You can specify a package name as a sequence of names separated by periods. For example, you might have developed several collections of classes dealing with geometry, one dealing with 2D shapes and another with 3D shapes. In this case you might include the class `Sphere` in a package with the statement:

```
package Geometry.Shapes3D;
```

and the class for circles in a package using the statement:

```
package Geometry.Shapes2D;
```

In this situation, the packages are expected to be in the directories `Shapes3D` and `Shapes2D`, and both of these must be sub-directories of `Geometry`. In general, you can have as many names as you like separated by periods to identify a package, but the name must reflect the directory structure where the package is stored.



Compiling a Package

Compiling the classes in a package can be a bit tricky unless you are clear on how you go about it. We will illustrate what you need to do assuming you are using the JDK under Microsoft Windows. The path to the package directory has to be explicitly made known to the compiler, even when the current directory is the one containing the package. If we have stored the `Geometry` package source files in the directory with the path `C:\Beg Java Stuff\Geometry`, then the path to the `Geometry` directory is `C:\Beg Java Stuff`. You can tell the compiler about this path using the `-classpath` option on the command line. Assuming that the current directory is the `Geometry` directory, we could compile the `Line.java` source file with the command:

```
C:\JavaStuff\Geometry>javac -classpath "C:\Beg Java Stuff" Line.java
```

This will result in both the `Line.java` and `Point.java` files being compiled, since `Line.java` refers to the other class. Since the directory in the path contains spaces, we have to enclose the path between double quotes.

Accessing a Package

How you access a package, when you are compiling a program that uses the package, depends on where you have put it. There are a couple of options here. The first, but not the best, is to leave the `.class` files for the classes in the package in the directory with the package name.

Let's look at that before we go on to the second possibility.

With the `.class` files in the original package directory, either the path to your package must appear in the string set for the `CLASSPATH` environment variable, or you must use the `-classpath` option on the command line when you invoke the compiler or the interpreter. This overrides the `CLASSPATH` environment variable if it happens to be set. Note that it is up to you to make sure the classes in your package are in the right directory. Java will not prevent you from saving a file in a directory that is quite different from that appearing in the package statement. Of the two options here, using the `-classpath` option on the command line is preferable, because it sets the classpaths transiently each time, and can't interfere with anything you do subsequently. In any event we will look at both possibilities.

If you elect to use the `CLASSPATH` environment variable, it only needs to contain the paths to your packages. The standard packages supplied with Java do not need to be considered as the compiler and the interpreter can always find them. For example, you might set it under Windows 95 or 98 by adding the command,

```
set CLASSPATH=.;C:\MySource;C:\MyPackages
```

to your `autoexec.bat` file. Now the compiler and the interpreter will look for the directories containing your packages in the current directory, as specified by the period, and the directories `C:\MySource` and `C:\MyPackages`. Of course, you may have as many paths as you want defined in `CLASSPATH`. They just need to be separated by semi-colons under Windows.

Under Unix, the equivalent to this might be:

```
CLASSPATH=.:usr/local/mysource:usr/local/mypackages
```

If you are using the Sun Java Development Kit, you also specify where your packages can be found by using the `-classpath` option when you execute the Java compiler or the interpreter. This has the advantage that it only applies for the current compilation or execution so you can easily set it to suit each run. The command to compile `MyProgram.java` defining the classpath as in the environment variable above would be:

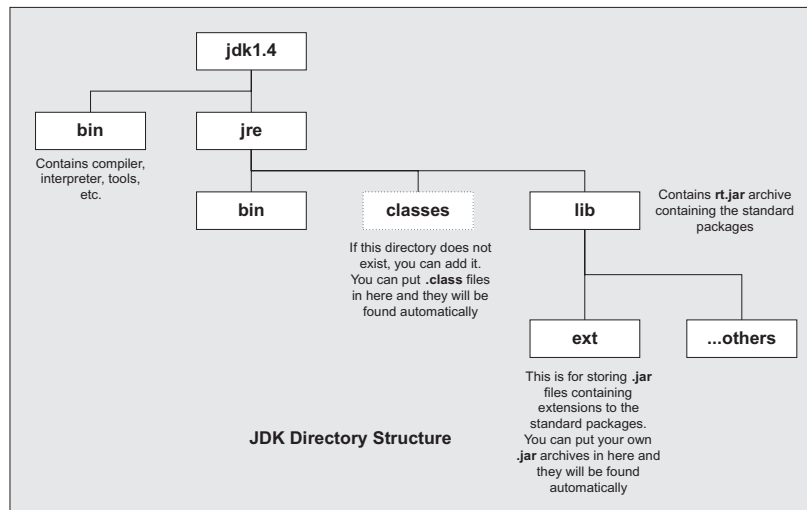
```
javac -classpath ".;C:\MySource;C:\MyPackages" MyProgram.java
```

If you don't set the classpath in one of these ways, or you set it incorrectly, Java will not be able to find the classes in any new packages you might create.

A new feature introduced with Java 2 provides you with a second way to make your packages available once you have compiled them. This provides a way of adding **extensions** to the set of standard packages.

Using Extensions

Extensions are `.jar` files stored within the `ext` directory that is created when you install the JDK. For more information on the use of the `JAR` tool to create `.jar` archives, see Appendix A. The default directory structure that is created is shown below.



The classes and packages in the `.jar` archives that you place in the `ext` directory will automatically be accessible when you compile or run your Java programs, without the need to set the `CLASSPATH` environment variable, or to use the `-classpath` command line option. When you create a `.jar` file for a package, you need to make sure that you add the `.class` files with the directory structure corresponding to the package name – you can't just add the `.class` files to the archive. For example, suppose we want to store our `Geometry` package in an archive. Assuming we have already compiled the package and the current directory contains the package directory, the following command can be used to create the archive:

```
C:\JavaStuff>jar cvf Geometry.jar Geometry\*.class
```

This will create the archive `Geometry.jar`, and add all the `.class` files that are in the `Geometry` directory to it. All you now need to do, to make the package available to any program that needs it is to copy it, to the `ext` directory in the JDK directory hierarchy.

The diagram above also shows the `classes` directory, which may not be created by default. You can put `.class` files in this directory and they will be found automatically when a program uses the classes they contain.

Adding Classes from a Package to Your Program

Assuming they have been defined with the `public` keyword, you can add all or any of the classes in a package to the code in your program by using an **import statement**. You can reference the classes that you make available to your program using the import statement just by using the class names. For example, to make available all the classes in the package `Geometry.Shapes3D` to a source file, you just need to add the following import statement to the beginning of the file:

```
import Geometry.Shapes3D.*; // Include all classes from this package
```

The keyword `import` is followed by the specification of what you want to import. The wildcard `*`, following the period after the package name, selects all the classes in the package, rather like selecting all the files in a directory. Now you can refer to any public class in the package just by using the class name. Again, the names of other classes in your program must be different from the names of the classes in the package.

If you want to add a particular class rather than an entire package, you specify its name explicitly in the `import` statement:

```
import Geometry.Shapes3D.Sphere; // Include the class Sphere
```

This includes only the `Sphere` class into the source file. By using a separate import statement for each individual class from the package, you can ensure that your source file only includes the classes that you need. This reduces the likelihood of name conflicts with your own classes, particularly if you are not fully familiar with the contents of the package and it contains a large number of classes.

Note that the `*` can only be used to select all the classes in a package. You can't use `Geometry.*` to select all the packages in the directory `Geometry`.

Packages and Names in Your Programs

A package creates a self-contained environment for naming your classes. This is the primary reason for having packages in Java. You can specify the names for classes in one package without worrying about whether the same names have been used elsewhere. Java makes this possible by treating the package name as part of the class name – actually as a prefix. This means that the class `Sphere` in the package `Geometry.Shapes3D` has the full name `Geometry.Shapes3D.Sphere`. If you don't use an import statement to incorporate the class in your program, you can still make use of the class by referring to it using its full class name. If you needed to do this with the class `Sphere`, you might declare a variable with the statement:

```
Geometry.Shapes3D.Sphere ball = new Geometry.Shapes3D.Sphere(10.0, 1.0, 1.0, 1.0);
```

While this is rather verbose, and certainly doesn't help the readability of the program, it does ensure there will be no conflict between this class and any other `Sphere` class that might be part of your program. You can usually contrive that your class names do not conflict with those in the commonly used standard Java packages, but in cases where you can't manage this, you can always fall back on using fully qualified class names. Indeed, there are occasions when you have to do this. This is necessary when you are using two different classes from different packages that share the same basic class name.

Standard Packages

All of the standard classes that are provided with Java are stored in standard packages. There is a substantial (more than 130 in SDK1.4) and growing list of standard packages but some of the ones you may hear about quite frequently are:

<code>java.lang</code>	Contains classes that are fundamental to Java (e.g. the Math class) and all of these are available in your programs automatically. You do not need an import statement to include them.
<code>java.io</code>	Contains classes supporting stream input/output operations.
<code>java.nio</code>	Contains classes supporting new input/output operations in JDK1.4 – especially with files.
<code>java.nio.channels</code>	Contains more classes supporting new input/output operations in JDK1.4 – the ones that actually read and write files.
<code>java.awt</code>	Contains classes that support Java's Graphical User Interface (GUI). While you can use these classes for GUI programming, it is almost always easier and better to use the alternative Swing classes.
<code>javax.swing</code>	Provides classes supporting the 'Swing' GUI components. These are not only more flexible and easier to use than the <code>java.awt</code> equivalents, but they are also implemented largely in Java with minimal dependency on native code.
<code>javax.swing.border</code>	Classes to support generating borders around Swing components.
<code>javax.swing.event</code>	Classes supporting event handling for Swing components.
<code>java.awt.event</code>	Contains classes that support event handling.
<code>java.awt.geom</code>	Contains classes for drawing and operating with 2D geometric entities.
<code>java.awt.image</code>	Contains classes to support image processing.
<code>java.applet</code>	This contains classes that enable you to write applets – programs that are embedded in a web page.
<code>java.util</code>	This contains classes that support a range of standard operations for managing collections of data, accessing date and time information, and analyzing strings.

The standard packages and the classes they contain cover an enormous amount of ground, so even in a book of this size it is impossible to cover them all exhaustively. There are now many more classes in the standard packages with JDK1.4 than there are pages in this book. However, we will be applying some classes from all of the packages in the table above, plus one or two others besides, in later chapters of the book.

Standard Classes Encapsulating the Basic Data Types

You saw in the previous chapter that we have classes available that allow you to define objects that encapsulate each of the basic data types in Java. These classes are:

Boolean	Character	Byte
Short	Integer	Long
Float	Double	

These are all contained in the package `java.lang` along with quite a few other classes such as the `String` and `StringBuffer` classes that we saw in Chapter 4, and the `Math` class. Each of these classes encapsulates the corresponding basic type, and includes methods for manipulating and interrogating objects of the class, as well as a number of static methods that provide utility functions for the underlying basic types. Each of the classes corresponding to a numeric type provides a static `toString()` method to convert to a `String` object, as we saw in the last chapter. There is also a non-static `toString()` method in all of these classes that returns a `String` representation of a class object.

The classes encapsulating the numeric basic types each contain the static `final` constants `MAX_VALUE` and `MIN_VALUE` that define the maximum and minimum values that can be represented. The floating-point classes also define the constants `POSITIVE_INFINITY`, `NEGATIVE_INFINITY`, and `NaN` (stands for **N**ot **a** **N**umber as it is the result of `0/0`), so you can use these in comparisons. Alternatively, you can test floating point values with the static methods `isInfinite()` and `isNaN()` – you pass your variable as an argument, and the methods return `true` for an infinite value or the `NaN` value respectively. Remember that an infinite value can arise without necessarily dividing by zero. Any computation that results in an exponent that is too large to be represented will produce either `POSITIVE_INFINITY` or `NEGATIVE_INFINITY`.

Conversely there are methods to convert from a `String` to a basic type. For example, the static `parseInt()` member of the class `Integer` accepts a `String` representation of an integer as an argument, and returns the equivalent value as type `int`. An alternative version of this method accepts a second argument of type `int` that specifies the radix to be used. If the `String` object cannot be parsed for any reason, if it contains invalid characters for instance, the method will throw an exception of type `NumberFormatException`. All the standard classes define methods to parse strings – `parseShort()`, `parseByte()`, and `parseLong()`.

There are many other operations supported by these classes so it is well worth browsing the JDK documentation for them.

Controlling Access to Class Members

We have not yet discussed in any detail how accessible class members are outside a class. You know that from inside a static class method you can refer to any of the static members of the class, and a non-static method can refer to any member of the class. The degree to which variables and methods within one class are accessible from other classes is more complicated. It depends on what **access attributes** you have specified for the members of a class, and whether the classes are in the same package. This is why we had to understand packages first.

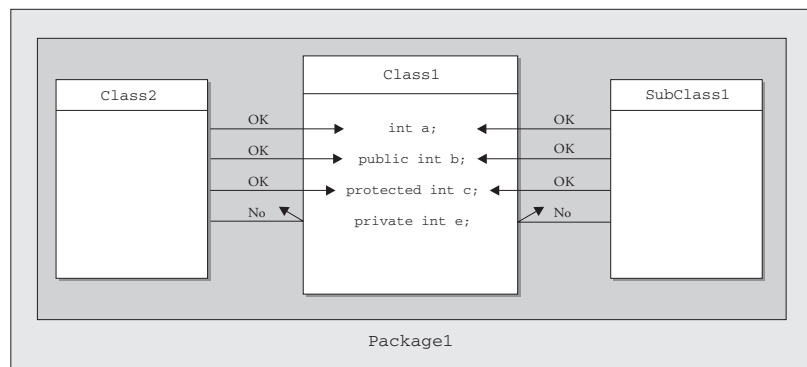
Using Access Attributes

Let's start by considering classes in the same package. Within a given package, any class has direct access to any other class name – for declaring variables or specifying method parameter types, for example – but the variables and methods that are members of that other class are not necessarily accessible. The accessibility of these is controlled by **access attributes**. You have four possibilities when specifying an access attribute for a class member, including what we have used in our examples so far – that is, not to specify anything at all – and each possibility has a different effect overall. The options you have for specifying the accessibility of a variable or a method in a class are:

Attribute	Permitted access
No access attribute	From methods in any class in the same package.
<code>public</code>	From methods in any class anywhere.
<code>private</code>	Only accessible from methods inside the class. No access from outside the class at all.
<code>protected</code>	From methods in any class in the same package and from any sub-class anywhere.

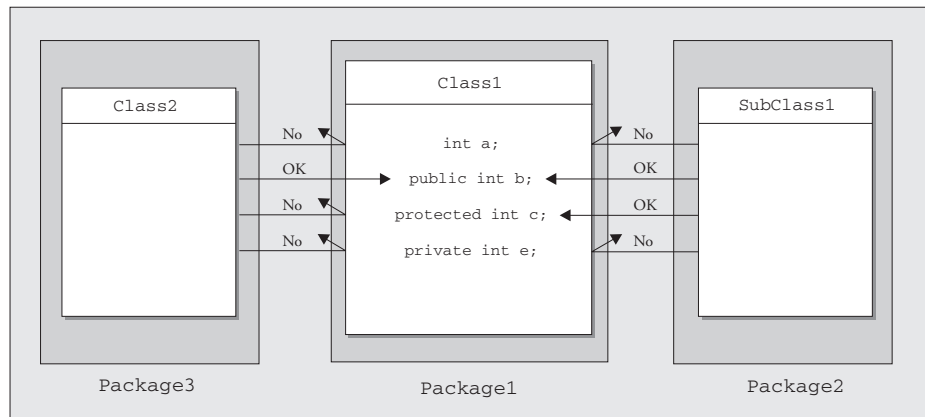
The table shows you how the access attributes you set for a class member determine the parts of the Java environment from which you can access it. We will discuss sub-classes in the next chapter, so don't worry about these for the moment. We will be coming back to how and when you use the `protected` attribute then. Note that `public`, `private`, and `protected` are all keywords. Specifying a member as `public` makes it completely accessible, and at the other extreme, making it `private` restricts access to members of the same class.

This may sound more complicated than it actually is. Look at the next diagram, which shows the access allowed between classes within the same package.



Within a package such as `package1`, only the `private` members of the class `Class1` can't be directly accessed by a method in another class in the same package. Declaring a class member to be `private` limits its availability solely to methods in the same class.

We saw earlier that a class definition must have an access attribute of `public` if it is to be accessible from outside the package. The next diagram shows the situation where the classes, seeking access to the members of a public class, are in different packages.



Here access is more restricted. The only members of Class1 that can be accessed from an ordinary class, Class2, in another package are those specified as `public`. Keep in mind that the class, Class1, must also have been defined with the attribute `public`. From a sub-class of Class1 that is in another package, the members of Class1, without an access attribute, cannot be reached, and neither can the `private` members – these can never be accessed externally under any circumstances.

Specifying Access Attributes

As you probably gathered from the diagrams that we just looked at, to specify an access attribute for a class member, you just add the keyword to the beginning of the declaration. Here is the `Point` class you saw earlier, but now with access attributes defined for its members:

Try It Out – Accessing the Point Class

Make the following changes to your `Point` class. If you save it in a new directory, do make sure `Line.java` is copied there as well. It will be useful later if they are in a directory with the name `Geometry`.

```
public class Point {
    // Create a point from its coordinates
    public Point(double xVal, double yVal) {
        x = xVal;
        y = yVal;
    }

    // Create a Point from an existing Point object
    public Point(final Point aPoint) {
        x = aPoint.x;
        y = aPoint.y;
    }
}
```

```

        // Move a point
    public void move(double xDelta, double yDelta) {
        // Parameter values are increments to the current coordinates
        x += xDelta;
        y += yDelta;
    }

    // Calculate the distance to another point
    public double distance(final Point aPoint) {
        return Math.sqrt((x - aPoint.x)*(x - aPoint.x)+(y - aPoint.y)*(y - aPoint.y));
    }

    // Convert a point to a string
    public String toString() {
        return Double.toString(x) + ", " + y;    // As "x, y"
    }

    // Coordinates of the point
    private double x;
    private double y;
}

```

The members have been re-sequenced within the class with the `private` members appearing last. You should maintain a consistent ordering of class members according to their access attributes, as it makes the code easier to follow. The ordering adopted most frequently is for the most accessible members to appear first, and the least accessible last, but a consistent order is more important than the particular order you choose.

How It Works

Now the instance variables `x` and `y` cannot be accessed or modified from outside the class as they are `private`. The only way these can be set or modified is through methods within the class, either with constructors, or the `move()` method. If it is necessary to obtain the values of `x` and `y` from outside the class, as it might well be in this case, a simple function would do the trick. For example:

```

    public double getX() {
        return x;
    }

```

Couldn't be easier really, could it? This makes `x` freely available, but prevents modification of its value from outside the class. In general, such methods are referred to as **accessor** methods, and usually have the form `getXXX()`. Methods that allow a private data member to be changed are called **mutator** methods, and are typically of the form `setXXX()` where a new value is passed as an argument. For example:

```

    public void setX(double inputX) {
        x = inputX;
    }

```

It may seem odd to use a method to alter the value of a `private` data member when you could just make it `public`. The main advantage of using a method in this way is that you can apply validity checks on the new value that is to be set.

Choosing Access Attributes

As you can see from the table of access attributes, all the classes we have defined so far have had members that are freely accessible within the same package. This applies both to the methods and the variables that were defined in the classes. This is not good object-oriented programming practice. As we said in Chapter 1, one of the ideas behind objects is to keep the data members encapsulated so they cannot be modified by all and sundry, even from other classes within the same package. On the other hand, the methods in your classes generally need to be accessible. They provide the outside interface to the class and define the set of operations that are possible with objects of the class. Therefore in the majority of situations with simple classes (i.e. no sub-classes), you should be explicitly specifying your class members as either `public` or `private`, rather than omitting the access attributes.

Broadly, unless you have good reasons for declaring them otherwise, the variables in a public class should be `private` and the methods that will be called from outside the class should be `public`. Even where access to the values of the variables from outside a class is necessary, you don't need to make them `public` or leave them without an access attribute. As we've just seen, you can provide access quite easily by adding a simple `public` method to return the value of a data member.

Of course, there are always exceptions:

- ❑ For classes in a package that are not public, and therefore not accessible outside the package, it may sometimes be convenient to allow other classes in the package direct access to the data members.
- ❑ If you have data members that have been specified as `final` so that their values are fixed, and they are likely to be useful outside the class, you might as well declare them to be `public`.
- ❑ You may well have methods in a class that are only intended to be used internally by other methods in the class. In this case you should specify these as `private`.
- ❑ In a class like the standard class, `Math`, which is just a convenient container for utility functions and standard data values, you will want to make everything `public`.

All of this applies to simple classes. We will see in the next chapter, when we will be looking at sub-classes, that there are some further aspects of class structure that you must take into account.

Using a Package and Access Attributes

Let's put together an example that uses a package that we will create. We could put the `Point` and `Line` classes that we defined earlier in a package we could call `Geometry`. We can then write a program that will import these classes and test them.

Try It Out – Packaging Up the Line and Point Classes

The source and `.class` files for each class in the package must be in a directory with the name `Geometry`. Remember that you need to ensure the path to the directory (or directories if you are storing `.class` files separately) `Geometry` appears in the `CLASSPATH` environment variable setting before you try compile or use either of these two classes. You can do this by specifying the `-classpath` option when you run the compiler or the interpreter.

To include the class `Point` in the package, the code in `Point.java` will be:

```
package Geometry;

public class Point {

    // Create a point from its coordinates
    public Point(double xVal, double yVal) {
        x = xVal;
        y = yVal;
    }

    // Create a Point from an existing Point object
    public Point(final Point aPoint) {
        x = aPoint.x;
        y = aPoint.y;
    }

    // Move a point
    public void move(double xDelta, double yDelta) {
        // Parameter values are increments to the current coordinates
        x += xDelta;
        y += yDelta;
    }

    // Calculate the distance to another point
    public double distance(final Point aPoint) {
        return Math.sqrt((x - aPoint.x)*(x - aPoint.x)+(y - aPoint.y)*(y - aPoint.y));
    }

    // Convert a point to a string
    public String toString() {
        return Double.toString(x) + ", " + y;    // As "x, y"
    }

    // Retrieve the x coordinate
    public double getX() {
        return x;
    }

    // Retrieve the y coordinate
    public double getY() {
        return y;
    }

    // Set the x coordinate
    public void setX(double inputX) {
        x = inputX;
    }

    // Set the y coordinate
    public void setY(double inputY) {
        y = inputY;
    }

    // Coordinates of the point
    private double x;
    private double y;
}
```

Note that we have added the `getX()`, `getY()`, `setX()` and `setY()` methods to the class to make the private data members accessible.

The `Line` class also needs to be amended to make the methods public and to declare the class as public. We also need to change its `intersects()` method so that it can access the private data members of `Point` objects using the `set...()` and `get...()` methods in the `Point` class. The code in `Line.java`, with changes highlighted, will be:

```
package Geometry;

public class Line {

    // Create a line from two points
    public Line(final Point start, final Point end) {
        this.start = new Point(start);
        this.end = new Point(end);
    }

    // Create a line from two coordinate pairs
    public Line(double xStart, double yStart, double xEnd, double yEnd) {
        start = new Point(xStart, yStart);    // Create the start point
        end = new Point(xEnd, yEnd);          // Create the end point
    }

    // Calculate the length of a line
    public double length() {
        return start.distance(end);           // Use the method from the Point class
    }

    // Return a point as the intersection of two lines -- called from a Line object
    public Point intersects(final Line line1) {

        Point localPoint = new Point(0, 0);

        double num =(this.end.getY() - this.start.getY())
            * (this.start.getX()-line1.start.getX())
            - (this.end.getX() - this.start.getX())
            * (this.start.getY() - line1.start.getY());

        double denom = (this.end.getY() - this.start.getY())
            * (line1.end.getX() - line1.start.getX())
            - (this.end.getX() - this.start.getX())
            * (line1.end.getY() - line1.start.getY());

        localPoint.setX(line1.start.getX() + (line1.end.getX() -
            line1.start.getX())*num/denom);
        localPoint.setY(line1.start.getY() + (line1.end.getY() -
            line1.start.getY())*num/denom);

        return localPoint;
    }

    // Convert a line to a string
    public String toString() {
        return "(" + start + "):(" + end + ")";    // As "(start):(end)"
    }                                           // that is, "(x1, y1):(x2, y2)"

    // Data members
```

```

    Point start;                                // Start point of line
    Point end;                                  // End point of line
}

```

Here we have left the data members without an access attribute, so they are accessible from the `Point` class, but not from classes outside the `Geometry` package.

How It Works

The package statement at the beginning of each source file defines the package to which the class belongs. Remember, you still have to save it in the correct directory, `Geometry`. Without the `public` attribute, the classes would not be available to classes outside the `Geometry` package.

Since we have declared the data members in the class `Point` as `private`, they will not be accessible directly. We have added the methods `getX()`, `getY()`, `setX()`, and `setY()` to the `Point` class to make the values accessible to any class that needs them.

The `Line` class hasn't been updated since our first example, so we first have to sort out the access attributes. The two instance variables are declared as before, without any access attribute, so they can be accessed from within the package but not from classes outside the package. This is an instance where exposing the data members within the package is very convenient, and we can do it without exposing the data members to any classes using the package. And we have updated the `intersects()` method to reflect the changes in accessibility made to the members of the `Point` class.

We can now write the program that is going to import and use the package that we have just created.

Try It Out – Testing the Geometry Package

We can create a succession of points, and a line joining each pair of successive points in the sequence, and then calculate the total line length.

```

import Geometry.*;    // Import the Point and Line classes

public class TryPackage {
    public static void main(String[] args) {
        double[][] coords = { {1.0, 0.0}, {6.0, 0.0}, {6.0, 10.0},
                                {10.0,10.0}, {10.0, -14.0}, {8.0, -14.0}};
        // Create an array of points and fill it with Point objects
        Point[] points = new Point[coords.length];
        for(int i = 0; i < coords.length; i++)
            points[i] = new Point(coords[i][0],coords[i][1]);

        // Create an array of lines and fill it using Point pairs
        Line[] lines = new Line[points.length - 1];
        double totalLength = 0.0;           // Store total line length here
        for(int i = 0; i < points.length - 1; i++) {
            lines[i] = new Line(points[i], points[i+1]); // Create a Line
            totalLength += lines[i].length();           // Add its length
            System.out.println("Line " + (i+1) + " " + lines[i] +
                               " Length is " + lines[i].length());
        }
    }
}

```

```
// Output the total length
System.out.println("\nTotal line length = " + totalLength);
}
}
```

You should save this as `TryPackage.java` in the directory `TryPackage`. If the path to your `Geometry` directory on a PC running Windows is `C:\Packages\Geometry`, you can compile this with the command:

```
javac -classpath ".;C:\Packages" TryPackage.java
```

This assumes the current directory contains the `TryPackage.java` file. The `-classpath` option specifies two paths separated by a semi-colon. The first path, specified by a period, is the current directory. This is necessary to enable the `TryPackage.java` source file to be found. The second path is `C:\Packages`, which is the directory containing our `Geometry` package. Without this the compiler will not be able to find the classes in the `Geometry` package and the compilation will fail.

Once you have a successful compilation, you can execute the program with the command:

```
java -classpath ".;C:\Packages" TryPackage
```

When the program executes, you should see the following output:

```
Line 1 (1.0, 0.0):(6.0, 0.0)   Length is 5.0
Line 2 (6.0, 0.0):(6.0, 10.0)  Length is 10.0
Line 3 (6.0, 10.0):(10.0, 10.0) Length is 4.0
Line 4 (10.0, 10.0):(10.0, -14.0) Length is 24.0
Line 5 (10.0, -14.0):(8.0, -14.0) Length is 2.0

Total line length = 45.0
```

How It Works

This example is a handy review of how you can define arrays, and also shows that you can declare an array of objects in the same way as you declare an array of one of the basic types. The dimensions of the array of arrays, `coords`, are determined by the initial values that are specified between the braces. The number of values within the outer braces determines the first dimension. Each of the elements in the array is itself an array of length two, with each pair of element values being enclosed within their own braces.

Since there are six sets of these, we have an array of six elements, each of which is itself an array of two elements. Each of these elements correspond to the (x, y) coordinates of a point.

You can see from this that, if necessary, you can create an array of arrays with each row having a different number of elements. The number of initializing values that appear, so they could all be different in the most general case, determines the length of each row.

We declare an array of `Point` objects with the same length as the number of (x, y) pairs in the `coords` array. This array is filled with `Point` objects in the `for` loop, which we created using the pairs of coordinate values from the `coords` array.

Since each pair of `Point` objects will define a `Line` object, we need one less element in the array `lines` than we have in the `points` array. We create the elements of the `lines` array in the second `for` loop using successive `Point` objects, and accumulate the total length of all the line segments by adding the length of each `Line` object to `totalLength` as it is created. On each iteration of the `for` loop, we output the details of the current line. Finally, we output the value of `totalLength`, which in this case is 45.

Note that the `import` statement adds the classes from the package `Geometry` to our program. These classes can be added to any application using the same `import` statement. You might like to try putting the classes in the `Geometry` package in a JAR file and try it out as an extension. Let's look at one other aspect of generating your own packages – compiling just the classes in the package without any program that makes use of them. We can demonstrate how this can be done on our `Geometry` package if you delete the `Line.class` and `Point.class` files from the package directory.

First make the directory, `C:\TryPackage`, that contains the package directory, `current`. Now you can compile just the classes in the `Geometry` package with the command:

```
javac -classpath "C:\TryPackage" Geometry/*.java
```

This will compile both the `Line` and `Point` classes so you should see the `.class` files restored in the `Geometry` directory. The files to be compiled are specified relative to the current directory as **`Geometry/*.java`**. Under Microsoft Windows this could equally well be **`Geometry\*.java`**. This specifies all files in the `Geometry` subdirectory to the current directory. The `classpath` must contain the path to the package directory, otherwise the compiler will not be able to find the package. We have defined it here using the `-classpath` option. We have not specified the current directory in `classpath` since we do not have any files there that need to be compiled. If we had included it in `classpath` it would not have made any difference – the classes in the `Geometry` package would compile just the same.

Nested Classes

All the classes we have defined so far have been separate from each other – each stored away in its own file. Not all classes have to be defined like this. You can put the definition of one class inside the definition of another class. The inside class is called a **nested class**. A nested class can itself have another class nested inside it, if need be.

When you define a nested class, it is a member of the enclosing class in much the same way as other members. A nested class can have an access attribute just like other class members, and the accessibility from outside the enclosing class is determined by the attributes in the same way.

```
public class Outside {

    // Nested class
    public class Inside {
        // Details of Inside class...
    }

    // More members of Outside class...
}
```

Here the class `Inside` is nested inside the class `Outside`. The `Inside` class is declared as a public member of `Outside`, so it is accessible from outside `Outside`. Obviously a nested class should have some specific association with the enclosing class. Arbitrarily nesting one class inside another would not be sensible. The enclosing class here is referred to as a **top-level class**. A top-level class is a class that contains a nested class but is not itself a nested class.

Our nested class here only has meaning in the context of an object of type `Outside`. This is because `Inside` is not declared as a static member of the class `Outside`. Until an object of type `Outside` has been created, you can't create any `Inside` objects. However, when you declare an object of a class containing a nested class, no objects of the nested class are necessarily created – unless of course the enclosing class's constructor creates them. For example, suppose we create an object with the following statement:

```
Outside outer = new Outside();
```

No objects of the nested class, `Inside`, are created. If you now wish to create an object of the type of the nested class you must refer to the nested class type using the name of the enclosing class as a qualifier. For instance, having declared an object of type `Outside`, we can create an object of type `Inside` as follows:

```
Outside.Inside inner = outer.new Inside();    // Define a nested class object
```

Here we have created an object of the nested class type that is associated with the object, `outer`, created earlier. We are creating an object of type `Inside` in the context of the object `outer`. Within non-static methods that are members of `Outside`, you can use the class name `Inside` without any qualification as it will be automatically qualified by the compiler with the `this` variable. So we could create a new `Inside` object from within the method of the object `Outside`:

```
Inside inner = new Inside();    // Define a nested class object
```

which is equivalent to:

```
this.Inside inner = this.new Inside();    // Define a nested class object
```

All this implies that a static method cannot create objects of a non-static nested class type. Because the `Inside` class is not a static member of the `Outside` class, such a member could refer to an object which does not exist – an error, if there are no `Inside` objects extant in the context of an `Outside` object. And as `Inside` is not a static data member of the `Outside` class, if a static method in the `Outside` class tried to create an object of type `Inside` directly, without first invoking an object of type `Outside`, it would be trying to create an object outside of that object's legitimate scope – an illegal maneuver.

Further, because the class `Inside` is not a static member of the `Outside` class, it cannot in turn contain any static data members itself. Since `Inside` is not static, it cannot act as a freestanding class with static members – this would be a logical contradiction.

Nested classes are typically used to define objects that at least have a strong association with objects of the enclosing class type, and often there is a tight coupling between the two. A further use for nested classes is for grouping a set of related classes under the umbrella of an enclosing class. We will be using this approach in examples later on in the book.

Static Nested Classes

To make objects of a nested class type independent of objects of the enclosing class, you can declare the nested class as `static`. For example,

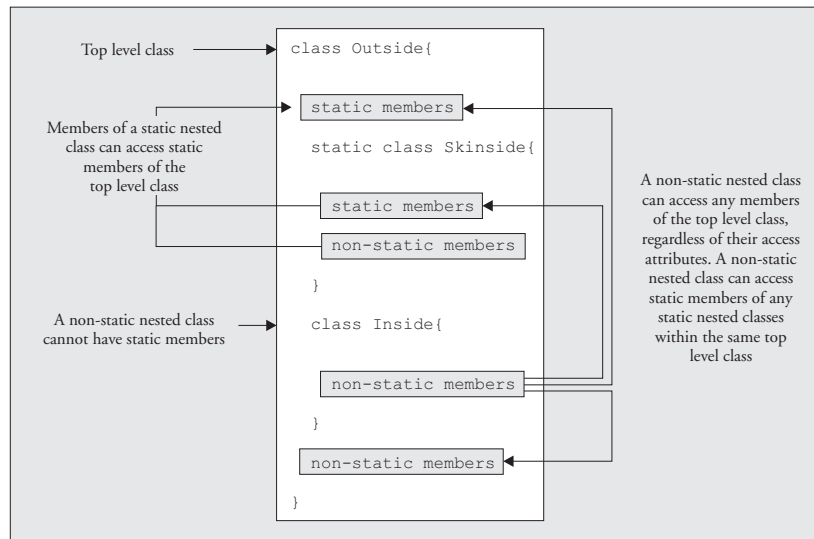
```
public class Outside {
    public static class Skinside {
        // Details of Skinside
    }

    // Nested class
    public class Inside {
        // Details of Inside class...
    }
    // More members of Outside class...
}
```

Now with `Skinside` inside `Outside` declared as `static`, we can declare objects of this nested class, independent from objects of `Outside`, and regardless of whether we have created any `Outside` objects or not. For example:

```
Outside.Skinside example = new Outside.Skinside();
```

This is significantly different from what we needed to do for a non-static nested class. Now we must use the nested class name qualified by the enclosing class name as the type for creating the object. Note that a static nested class can have static members, whereas a non-static nested class cannot.



Let's see how a nested class works in practice with a simple example. We will create a class `MagicHat` that will define an object containing a variable number of rabbits. We will put the definition for the class `Rabbit` inside the definition of the class `MagicHat`. The basic structure of `MagicHat.java` will be:

```
public class MagicHat {
    // Definition of the MagicHat class...

    // Nested class to define a rabbit
    static class Rabbit {
        // Definition of the Rabbit class...
    }
}
```

Here the nested class is defined as static because we want to be able to have static members of this class. We will see a little later how it might work with a non-static nested class.

Try It Out – Rabbits Out of Hats

Let's add the detail of the MagicHat class definition:

```
import java.util.Random;                // Import Random class

public class MagicHat {
    static int maxRabbits = 5;           // Maximum rabbits in a hat
    static Random select = new Random(); // Random number generator

    // Constructor for a hat
    public MagicHat(String hatName) {
        this.hatName = hatName;         // Store the hat name
        rabbits = new Rabbit[1+select.nextInt(maxRabbits)]; // Random rabbits

        for(int i = 0; i < rabbits.length; i++)
            rabbits[i] = new Rabbit();   // Create the rabbits
    }

    // String representation of a hat
    public String toString() {
        // Hat name first...
        String hatString = "\n" + hatName + " contains:\n";

        for(int i = 0; i < rabbits.length; i++)
            hatString += "\t" + rabbits[i] + " "; // Add the rabbits strings
        return hatString;
    }

    private String hatName;              // Name of the hat
    private Rabbit rabbits[];            // Rabbits in the hat

    // Nested class to define a rabbit
    static class Rabbit {
        // Definition of the Rabbit class...
    }
}
```

Instead of the old `Math.random()` method that we have been using up to now to generate pseudo-random values, we are using an object of the class `Random` that is defined in the package `java.util`. An object of type `Random` has a variety of methods to generate pseudo-random values of different types, and with different ranges. The method `nextInt()` that we are using here returns an integer that is zero or greater, but less than the integer value you pass as an argument. Thus if you pass the length of an array to it, it will generate a random index value that will always be legal for the array size.

We can now add the definition of the `Rabbit` class. When we create a `Rabbit` object, we want it to have a unique name so we can distinguish one `Rabbit` from another. We can generate unique names by selecting one from a limited set of fixed names, and then appending an integer that is different each time the base name is used. Here's what we need to add for the `Rabbit` class definition:

```
public class MagicHat {

    // Definition of the MagicHat class - as before...

    // Nested class to define a rabbit
    static class Rabbit {
        // A name is a rabbit name from rabbitNames followed by an integer
        static private String[] rabbitNames = {"Floppsy", "Moppsy",
                                                "Gnasher", "Thumper"};
        static private int[] rabbitNamesCount = new int[rabbitNames.length];
        private String name;                                // Name of the rabbit

        // Constructor for a rabbit
        public Rabbit() {
            int index = select.nextInt(rabbitNames.length); // Get random name index
            name = rabbitNames[index] + (++rabbitNamesCount[index]);
        }

        // String representation of a rabbit
        public String toString() {
            return name;
        }
    }
}
```

Note that the constructor in the `Rabbit` class can access the `select` member of the enclosing class `MagicHat`, without qualification. This is only possible with static members of the enclosing class – you can't refer to non-static members of the enclosing class here because there is no object of type `MagicHat` associated with it.

We can use the following application class to try out our nested class:

```
public class TryNestedClass {
    static public void main(String[] args) {
        // Create three magic hats and output them
        System.out.println(new MagicHat("Gray Topper"));
        System.out.println(new MagicHat("Black Topper"));
        System.out.println(new MagicHat("Baseball Cap"));
    }
}
```

When I ran the program, I got the output:

```
Gray Topper contains:
    Moppsy1      Moppsy2      Floppsy1

Black Topper contains:
    Thumper1     Moppsy3      Thumper2     Gnasher1

Baseball Cap contains:
    Moppsy4      Moppsy5      Thumper3
```

You are likely to get something different.

How It Works

Each `MagicHat` object will contain a random number of `Rabbit` objects. The constructor for a `MagicHat` object stores the name of the hat in its private member `hatName`, and generates a `Rabbit` array with at least one, and up to `maxRabbits` elements. This is done with the expression `1+select.nextInt(maxRabbits)`. Calling `nextInt()` with the argument `maxRabbits` will return a value that is from 0 to `maxRabbits-1` inclusive. Adding 1 to this will result in a value from 1 to `maxRabbits` inclusive. The array so created is then filled with `Rabbit` objects.

The `MagicHat` class also has a method `toString()` method which returns a `String` object containing the name of the hat and the names of all the rabbits in the hat. This assumes that the `Rabbit` class also has a `toString()` method defined. We will be able to use the `toString()` implicitly in an output statement when we come to create and display `MagicHat` class objects.

The base names that we use to generate rabbit names are defined in the static array `rabbitNames[]` in the `Rabbit` class. The count for each base name, which we will append to the base name to produce a unique name for a rabbit, is stored in the static array `rabbitNamesCount[]`. This has the same number of elements as the `rabbitNames` array, and each element stores a value to be appended to the corresponding name in the `rabbitNames` array. The `Rabbit` class has the data member, `name`, to store a name that is initialized in the constructor. A random base name is selected from the `rabbitNames[]` array using an index value from 0 up to one less than the length of this array. We then append the current count for the name incremented by 1, so successive uses of any base name such as `Gnasher`, for example, will produce names `Gnasher1`, `Gnasher2`, and so on. The `toString()` method for the class returns the name for the `Rabbit` object.

The method `main()` in `TryNestedClass` creates three `MagicHat` objects and outputs the string representation of each of them. Putting the object as an argument to the `println()` method will call the `toString()` method for the object automatically, and the `String` object that is returned will be output to the screen.

Using a Non-Static Nested Class

In our previous example, we could define the `Rabbit` class as non-static by deleting the keyword `static`. However, if you try that, the program will no longer compile and run. The problem is the static data members' `rabbitNames` and `rabbitNamesCount` in the `Rabbit` class. We saw earlier that a non-static nested class cannot have static members, so we must seek an alternative way of dealing with names.

We could consider making these arrays non-static. This has several disadvantages. First, each `Rabbit` object would have its own copy of these arrays – an unnecessary duplication of data. A more serious problem is that our naming process would not work. Because each object has its own copy of the `rabbitNamesCount` array, the names generated are not going to be unique.

The answer is to keep `rabbitNames` and `rabbitNamesCount` as static, but put them in the `MagicHat` class instead. Let's see that working.

Try It Out – Accessing the Top Level Class Members

We need to modify the class definition to:

```
public class MagicHat {
    static int maxRabbits = 5;                // Maximum rabbits in a hat
    static Random select = new Random();      // Random number generator
    static private String[] rabbitNames = {"Floppsy", "Moppsy",
                                           "Gnasher", "Thumper"};
    static private int[] rabbitNamesCount = new int[rabbitNames.length];

    // Constructor for a hat
    public MagicHat(final String hatName) {
        this.hatName = hatName;              // Store the hat name
        rabbits = new Rabbit[1+select.nextInt(maxRabbits)]; // Random rabbits

        for(int i = 0; i < rabbits.length; i++)
            rabbits[i] = new Rabbit();        // Create the rabbits
    }

    // String representation of a hat
    public String toString() {
        // Hat name first...
        String hatString = "\n" + hatName + " contains:\n";

        for(int i = 0; i < rabbits.length; i++)
            hatString += "\t" + rabbits[i] + " "; // Add the rabbits strings
        return hatString;
    }
    private String hatName;                  // Name of the hat
    private Rabbit rabbits[];                // Rabbits in the hat

    // Nested class to define a rabbit
    class Rabbit {
        private String name;                 // Name of the rabbit

        // Constructor for a rabbit
        public Rabbit() {
            int index = select.nextInt(rabbitNames.length); // Get random name index
            name = rabbitNames[index] + (++rabbitNamesCount[index]);
        }

        // String representation of a rabbit
        public String toString() {
```

```
        return name;
    }
}
}
```

The only changes are the deletion of the static keyword in the definition of the `Rabbit` class – the data members relating to rabbit names have been moved to the `MagicHat` class. You can run this with the same version of `TryNestedClass` and it should produce output much the same as before.

How It Works

Although the output is much the same, what is happening is distinctly different. The `Rabbit` objects that are created in the `MagicHat` constructor are now associated with the current `MagicHat` object that is being constructed. The `Rabbit()` constructor call is actually `this.Rabbit()`.

Using a Nested Class outside the Top-Level Class

You can create objects of an inner class outside the top-level class containing the inner class. As we discussed, how you do this depends on whether the nested class is a static member of the enclosing class. With the first version of our `MagicHat` class, with a static `Rabbit` class, you could create an independent rabbit by adding the following statement to the end of `main()`:

```
System.out.println("An independent rabbit: " + new MagicHat.Rabbit());
```

This `Rabbit` object is completely free – there is no `MagicHat` object to restrain it. In the case of a non-static `Rabbit` class, things are different. Let's try this using a modified version of the previous program.

Try It Out – Free Range Rabbits (Almost)

We can see how this works by modifying the method `main()` in `TryNestedClass` to create another `MagicHat` object, and then create a `Rabbit` object for it:

```
static public void main(String[] args) {
    // Create three magic hats and output them
    System.out.println(new MagicHat("Gray Topper"));
    System.out.println(new MagicHat("Black Topper"));
    System.out.println(new MagicHat("Baseball Cap"));

    MagicHat oldHat = new MagicHat("Old hat");    // New hat object
    MagicHat.Rabbit rabbit = oldHat.new Rabbit(); // Create rabbit object
    System.out.println(oldHat);                  // Show the hat
    System.out.println("\nNew rabbit is: " + rabbit); // Display the rabbit
}
```

The output produced is:

```
Gray Topper contains:
    Thumper1

Black Topper contains:
    Moppsy1    Thumper2    Thumper3
```

```

Baseball Cap contains:
    Floppsy1    Floppsy2    Thumper4

Old hat contains:
    Floppsy3    Thumper5    Thumper6    Thumper7    Thumper8

New rabbit is: Thumper9

```

How It Works

The new code first creates a `MagicHat` object, `oldHat`. This will have its own rabbits. We then use this object to create an object of the class `MagicHat.Rabbit`. This is how a nested class type is referenced – with the top-level class name as a qualifier. You can only call the constructor for the nested class in this case by qualifying it with a `MagicHat` object name. This is because a non-static nested class can refer to members of the top-level class – including instance members. Therefore, an instance of the top-level class must exist for this to be possible.

Note how the top-level object is used in the constructor call. The object name qualifier goes before the keyword `new` which precedes the constructor call for the inner class. This creates an object, `rabbit`, in the context of the object `oldHat`. This doesn't mean `oldHat` has `rabbit` as a member. It means that, if top-level members are used in the inner class, they will be the members for `oldHat`. You can see from the example that the name of the new rabbit is not part of the `oldHat` object, although it is associated with `oldHat`. You could demonstrate this by modifying the `toString()` method in the `Rabbit` class to:

```

public String toString() {
    return name + " parent: "+hatName;
}

```

If you run the program again, you will see that when each `Rabbit` object is displayed, it will also show its parent hat.

Local Nested Classes

You can define a class inside a method – where it is called a **local nested class**. It is also referred to as a **local inner class**, since a non-static nested class is often referred to as an **inner class**. You can only create objects of a local inner class locally – that is, within the method in which the class definition appears. This is useful when the computation in a method requires the use of a specialized class that is not required or used elsewhere.

A local inner class can refer to variables declared in the method in which the definition appears, but only if they are `final`.

The finalize() Method

You have the option of including a method `finalize()` in a class definition. This method is called automatically by Java before an object is finally destroyed and the space it occupies in memory is released. Please note that this may be some time after the object is inaccessible in your program. When an object goes out of scope, it is dead as far as your program is concerned, but the Java Virtual Machine may not get around to disposing of the remains until later. When it does, it calls the `finalize()` method for the object. The form of the `finalize()` method is:

```
protected void finalize() {  
    // Your clean-up code...  
}
```

This method is useful if your class objects use resources that require some special action when they are destroyed. Typically these are resources that are not within the Java environment and not guaranteed to be released by the object itself. This means such things as graphics resources – fonts or other drawing related resources that are supplied by the host operating system, or external files on the hard disk. Leaving these around after an object is destroyed wastes system resources and, in some circumstances (with graphics resources under Windows 95 for instance) if you waste enough of them, your program, and possibly other programs the system is supporting, may stop working. For most classes this is not necessary, but if an object opened a disk file for example, but did not guarantee its closure, you would want to make sure that the file was closed when the object was destroyed. You can implement the `finalize()` method to take care of this.

Another use for the `finalize()` method is to record the fact that the object has been destroyed. We could implement the `finalize()` method for the `Sphere` class to decrement the value of the static member, `count`, for instance. This would make `count` a measure of how many `Sphere` objects were around, rather than how many had been created. It would, however, not be an accurate measure for reasons that we will come to in a moment.

You cannot rely on an object being destroyed when it is no longer available to your program code. Unless your program calls the `System.gc()` method, the Java Virtual Machine will only get rid of unwanted objects and free the memory they occupy if it runs out of memory, or if there is no activity within your program – for example when waiting for input. As a result objects may not get destroyed until execution of your program ends. You also have no guarantee as to when a `finalize()` method will be called. All you are assured is that it will be called before the memory that the object occupied is freed. Nothing time-sensitive should be left to the `finalize()` method.

One consequence of this is that there are circumstances where this can cause problems – when you don't allow for the possibility of your objects hanging around. For example, suppose you create an object in a method that opens a file, and rely on the `finalize()` method to close it. If you then call this method in a loop, you may end up with a large number of files open at one time, since the object that is created in each call of the method will not necessarily be destroyed immediately on return from the method. This introduces the possibility of your program attempting to have more files open simultaneously than the host operating system allows. In this situation, you should make sure a file is closed when you have finished with it, by including an object method to close it explicitly – for example `close()`.

The `System` class also provides another possible approach. You can suggest to the JVM that the `finalize()` methods for all discarded objects should be run, if they haven't been already. You just call the `runFinalization()` method:

```
System.runFinalization();
```

This is another of those 'best efforts' deals on the part of the JVM. It will do its very best to run `finalize()` for any dead objects that are laying around before return from the `runFinalization()` method, but like with a lot of things in this life, there are no guarantees.

Native Methods

It is possible to include a method in a class that is implemented in some other programming language, such as C or C++, external to the Java Virtual Machine. To specify such a method within a class definition you use the keyword `native` in the declaration of the method. For example:

```
public native long getData();    // Declare a method that is not in Java
```

Of course the method will have no body in Java since it is defined elsewhere, where all the work is done, so the declaration ends with a semi-colon. The implementation of a native method will need to use an interface to the Java environment. The standard API for implementing native methods in C for example, is called JNI – the **J**ava **N**ative **I**nterface.

The major drawback to using native methods in Java is that your program will no longer be portable. Security requirements for applets embedded in Web pages require that the code must all be written in Java – using native methods in an applet is simply not possible. Since the primary reasons for using Java are the portability of the code and the ability to produce applets, the need for you to add native methods to your Java programs will be minimal. We will, therefore, not delve any deeper into this topic.

Summary

In this chapter you have learned all the essentials of defining your own classes. You can now create your own class types to fit the context of the problems you are dealing with. We will build on this in the next chapter to enable you to add more flexibility to the operations on your class objects by showing you how to realize polymorphism.

The important points covered in this chapter are:

- ❑ A class definition specifies the variables and methods that are members of the class.
- ❑ Each class must be saved in a file with the same name as the class, and with the extension `.java`.
- ❑ Class variables are declared using the keyword `static`, and one instance of each class variable is shared amongst all objects of a class.
- ❑ Each object of a class will have its own instance variables – these are variables declared without using the keyword `static`.
- ❑ Methods that are specified as `static` can be called even if no class objects exist, but a `static` method cannot refer to instance variables.

- ❑ Methods that are not specified as `static` can access any of the variables in the class directly.
- ❑ Recursive methods are methods that call themselves.
- ❑ Access to members of a class is determined by the access attributes that are specified for each of them. These can be `public`, `private`, `protected`, `package private`, or nothing at all.
- ❑ Classes can be grouped into a package. If a class in a package is to be accessible from outside the package the class must be declared using the keyword `public`.
- ❑ To designate that a class is a member of a package you use a `package` statement at the beginning of the file containing the class definition.
- ❑ To add classes from a package to a file you use an `import` statement immediately following any `package` statement in the file.
- ❑ A native method is a method implemented in a language other than Java. Java programs containing native methods cannot be applets and are no longer portable.
- ❑ A field does not get ploughed by turning it over in your mind.

Exercises

1. Define a class for rectangle objects defined by two points, the top-left and bottom-right corners of the rectangle. Include a constructor to copy a rectangle, a method to return a rectangle object, that encloses the current object and the rectangle passed as an argument, and a method to display the defining points of a rectangle. Test the class by creating four rectangles, and combining these cumulatively, to end up with a rectangle enclosing them all. Output the defining points of all the rectangles you create.
2. Define a class, `mcmLength`, to represent a length measured in meters, centimeters, and millimeters, each stored as integers. Include methods to add and subtract objects, to multiply and divide an object by an integer value, to calculate an area resulting from the product of two objects, and to compare objects. Include constructors that accept: three arguments – meters, centimeters, and millimeters; one integer argument in millimeters; one `double` argument in centimeters and no arguments, which creates an object with the length set to zero. Check the class by creating some objects and testing the class operations.
3. Define a class, `tkgWeight`, to represent a weight in tons, kilograms, and grams, and include a similar range of methods and constructors as the previous example. Demonstrate this class by creating and combining some class objects.
4. Put both the previous classes in a package called `Measures`. Import this package into a program that will calculate and display the total weight of the following: 200 carpets – size: 4 meters by 2 meters 9 centimeters, that weigh 1.25 kilograms per square meter; and 60 carpets – size: 3 meters 57 centimeters by 5 meters, that weigh 1.05 kilograms per square meter.

